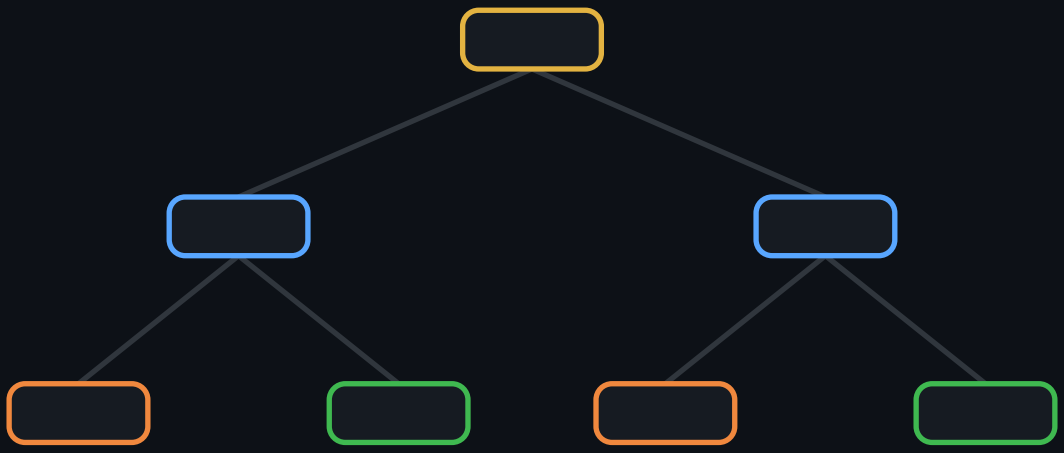


DECISION TREES

Hyperparameters, Grid Search & Pro Tips

From zero to pro — with pictures, math and worked examples

criterion · max_depth · min_samples_split · min_samples_leaf
max_features · ccp_alpha · GridSearchCV · ensembles

Dark-theme edition — made for comfortable reading

Welcome — read this first

This tutorial takes you from **“what even is a decision tree?”** to **tuning one like a professional**. Nothing is assumed: every idea is explained in plain language first, then shown with a picture, then (where it helps) with the exact math, and finally with real code you can run.

Everything from your original notes is included — the six hyperparameters, the grid-search table, the detailed explanations, the pro tips (both the technical and the simple-language versions), the complete code and its explanations. It has simply been re-organised into a learning path, and extended with the missing background (how a tree actually chooses its questions, how pruning works, how cross-validation works, and so on).

Level	Chapters	After this level you can…
BASIC	1 · What is a decision tree? 2 · How a tree learns (from scratch)	Read a tree, explain Gini / entropy, and compute a split by hand.
INTERMEDIATE	3 · Overfitting & hyperparameters 4 · The six hyperparameters one by one	Explain what every knob does and which direction makes the model simpler or more complex.
ADVANCED	5 · Grid search values & GridSearchCV 6 · The complete code, line by line	Run a cross-validated search and read its output.
PRO	7 · Pro tips & best practices 8 · Pro-level extras, cheat-sheet, glossary, exercises	Prepare data smartly, handle imbalance, know when to move to forests / boosting, and avoid classic mistakes.

How to read the coloured boxes

KEY IDEA — the one thing to remember.

ANALOGY — a real-life comparison.

MATH — the formula behind it (skip on first read if you like).

EXAMPLE — numbers you can verify by hand.

PRO TIP — what practitioners actually do.

WATCH OUT — common mistakes and traps.

IN PLAIN WORDS — the same idea in everyday language.

> **NOTE** **About the numbers in this tutorial**
>
> All pictures and results (accuracies, best parameters, tables) were **actually computed** for this document with scikit-learn on the data described next to each figure, so you can reproduce them. A few results may differ slightly on your machine or with another scikit-learn version — that is normal, and Chapter 6 explains why.

Table of contents

1 What is a Decision Tree? 5

1.1 The idea in one minute 5
1.2 Vocabulary you must know 6
1.3 Making a prediction: walk down the tree 6
1.4 The same tree seen as rectangles 6
1.5 Two flavours: classification and regression 7
1.6 Why people love (and sometimes leave) decision trees 7

2 How a Tree Learns — From Scratch 8

2.1 The learning recipe (recursive splitting) 8
2.2 Measuring “how mixed is this node?” 8
2.3 Scoring a split: weighted impurity and information gain 9
2.4 A complete worked example (10 customers) 9
2.5 Regression trees 12
2.6 When does a branch stop growing? 12

3 Overfitting, Underfitting & the Idea of Hyperparameters 13

3.1 Parameters vs. hyperparameters 13
3.2 Overfitting and underfitting 13
3.3 Bias–variance trade-off (the math behind the U-curve) 14
3.4 Why we hold back a test set 14

4 The Six Hyperparameters, One by One 15

4.1 The big picture in one table 15
4.2 criterion 15
4.3 max_depth 16
4.4 min_samples_split 17
4.5 min_samples_leaf 17
4.6 max_features 19
4.7 ccp_alpha 20
4.8 Other parameters worth knowing (extras) 21

5 Grid Search: Finding the Best Settings Automatically 22

5.1 Recommended grid-search values (your table) 22
5.2 What is a grid search? 22
5.3 How each combination is scored: cross-validation 22
5.4 Reading the result: a heat-map of the search 23
5.5 When the grid gets too big 24

6 The Complete Code, Line by Line 25

6.1 The data and the goal 25

6.2 The full program 26

6.3 What each step does 27

6.4 Four lines worth understanding deeply 27

6.5 The real output 27

6.6 How to read it 28

7 Pro Tips & Best Practices 30

7.1 Use cost-complexity pruning (ccp_alpha) 30

7.2 Skip feature scaling — and create smart data instead 30

7.3 Balance class weights 31

7.4 Limit feature cardinality 32

7.5 Drop highly correlated features 32

7.6 Remove noisy variables early (“junk data”) 33

7.7 Hide some data to force creative thinking (max_features) 34

7.8 Put your whole computer to work (n_jobs=-1) 34

7.9 Accept that one tree is never enough (ensembles) 34

8 Pro-Level Extras, Cheat-Sheet & Practice 36

8.1 A professional workflow in eight steps 36

8.2 Putting preprocessing inside a Pipeline (no leakage) 36

8.3 Interpreting a tree 36

8.4 Choosing the scoring metric 37

8.5 Classic mistakes 37

8.6 Cheat-sheet: symptom → remedy 37

8.7 Glossary 38

8.8 Practice exercises 38

BASIC · CHAPTER 1

1 What is a Decision Tree?

The model that plays “20 Questions” with your data.

1.1 The idea in one minute

A **decision tree** is a flow-chart of yes/no questions that was *learned automatically from data*. To make a prediction you start at the top, answer one question after another, and follow the arrows until you reach an answer.

> **ANALOGY The game of 20 Questions**
> Someone thinks of an animal. You ask: *“Does it live in water?”* → *“Does it have fur?”* → … Each answer shrinks the list of possibilities until you can name the animal. A decision tree plays exactly this game, except that it **chooses the best questions by itself** by studying thousands of examples.

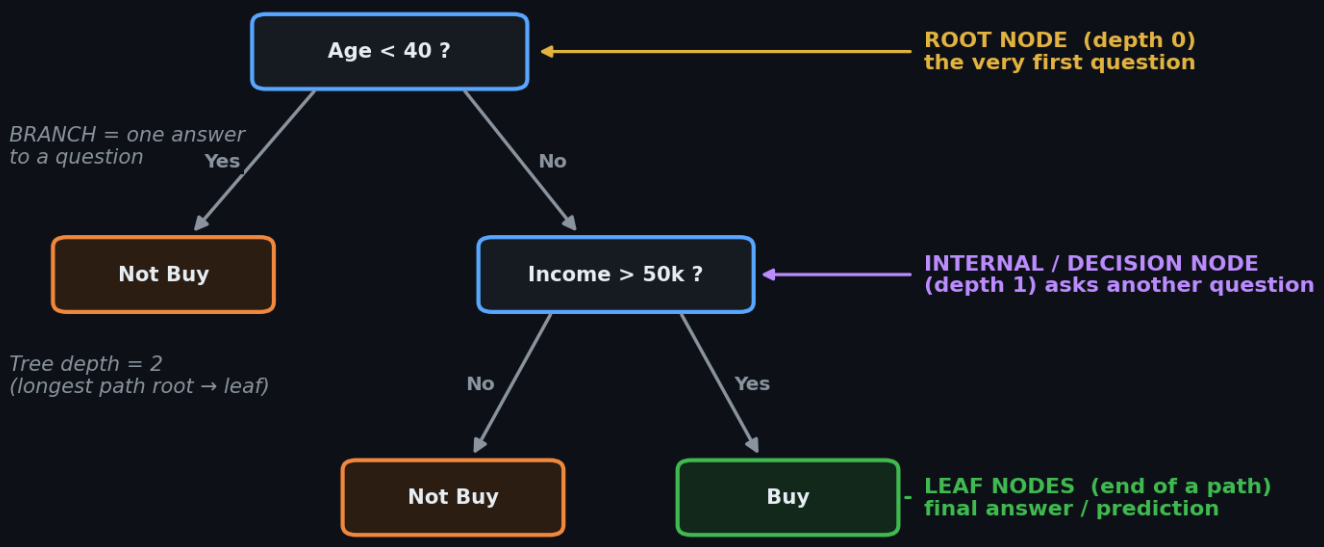


Figure 1.1 — The anatomy of a tree: a tiny model that predicts whether a customer will buy.

1.2 Vocabulary you must know

Term	Meaning	In Figure 1.1
Root node	The first question; it sees *all* the data.	“Age < 40 ?”
Internal (decision) node	Any node that asks a question and sends data left or right.	“Income > 50k ?”
Branch	One possible answer to a question (an arrow).	“Yes” / “No” arrows
Leaf (terminal) node	A node that does not split any more; it stores the final prediction.	“Buy”, “Not Buy”
Depth	Number of questions on the longest path from root to a leaf.	2
Parent / child	A node that splits is the *parent* of the two nodes it creates.	root is parent of two nodes
Split / threshold	The question itself: *feature* + *cut-off value*.	feature=Age, threshold=40
Impurity	How mixed the classes are inside a node (0 = perfectly pure). Chapter 2.	—

1.3 Making a prediction: walk down the tree

EXAMPLE **Customer: age 35, income 60 000**

Step 1. Root asks *Age < 40?* → 35 < 40 is **Yes** → go left.
Step 2. We arrive at a leaf labelled **Not Buy**. Prediction done — only one question was needed.
Another customer: age 52, income 80 000. Root: 52 < 40? **No** → right. Next: *Income > 50k?* → **Yes** → leaf **Buy**.

1.4 The same tree seen as rectangles

Every question cuts the data space with a **straight, axis-parallel line** (“x1 < 5”). After several questions the space is divided into **rectangles**, and each rectangle (= each leaf) predicts one answer. This picture is the key to understanding overfitting later: more questions → more and smaller rectangles → a more complicated boundary.

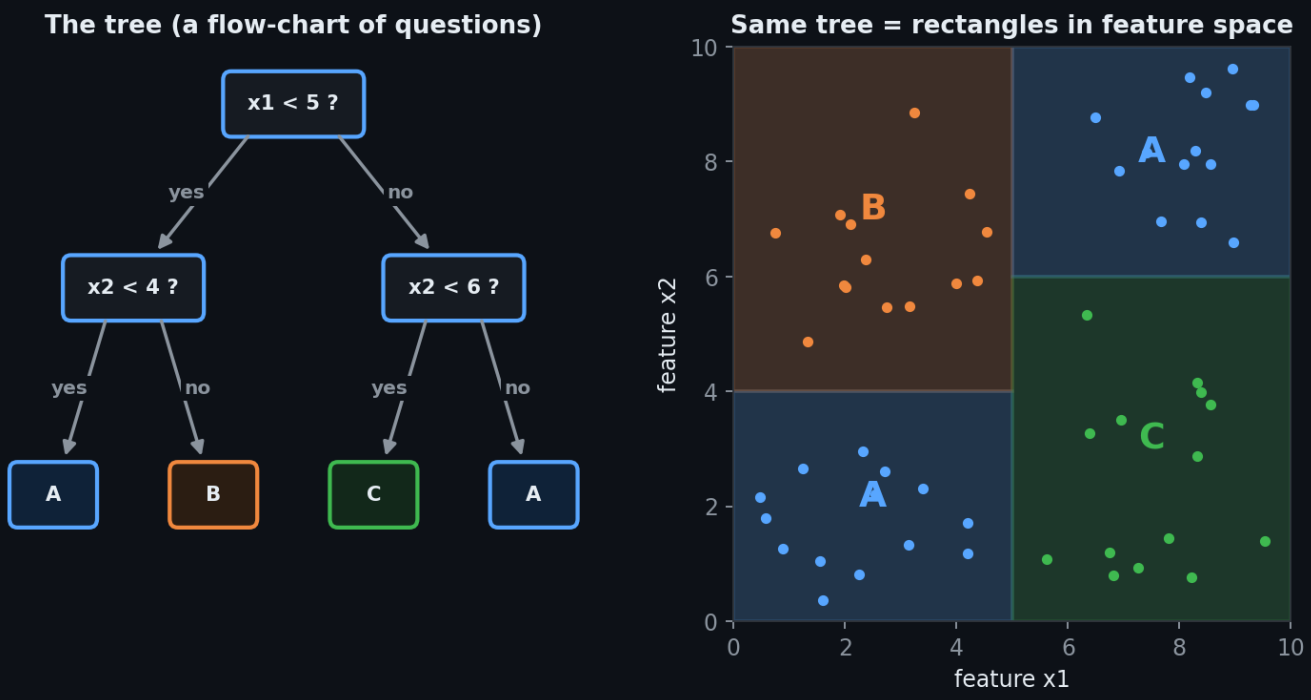


Figure 1.2 — Left: the tree. Right: the same tree as rectangles in a 2-feature space.

1.5 Two flavours: classification and regression

	Classification tree	Regression tree
Predicts	a **category** (spam / not spam, malignant / benign)	a **number** (house price, temperature)
Leaf stores	the **majority class** of the training rows in that leaf (and class probabilities)	the **average** (or median) of the target values in that leaf
sklearn class	`DecisionTreeClassifier`	`DecisionTreeRegressor`
Impurity measure	Gini or Entropy	Squared error (MSE) or Absolute error (MAE)

1.6 Why people love (and sometimes leave) decision trees

Strengths	Weaknesses
Easy to understand and to explain to non-experts	Grow too deep → memorise the training data (overfit)
No need to scale or normalise features (Chapter 7)	Unstable: a small change in data can change the whole tree
Handle numbers and (encoded) categories, non-linear patterns and interactions	Only axis-parallel cuts → diagonal patterns need many splits
Fast to train and to predict	A single tree has a ceiling on accuracy → forests / boosting

Your first three lines of scikit-learn — notice that `max_depth` is already a **hyperparameter**, the main topic of this tutorial:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=3)   # max_depth = a HYPERPARAMETER (you choose it)
model.fit(X_train, y_train)                   # the tree LEARNS its questions from data
model.predict(X_new)                          # prediction = walk down the tree
```

BASIC · CHAPTER 2

2 How a Tree Learns — From Scratch

Where do the questions come from? Impurity, information gain and a worked example you can check with a calculator.

2.1 The learning recipe (recursive splitting)

Training a tree is a surprisingly short recipe, repeated again and again:

- **Start** with all training rows in one node (the root).
- **Try every possible question**: for each feature, for each possible cut-off value.
- **Score each question** by how much *purer* it makes the two resulting groups.
- **Keep the best question**, split the rows into a left group and a right group.
- **Repeat** steps 2–4 inside each new group (this is called *recursion*).
- **Stop** a branch when a stopping rule says so (pure node, depth limit, too few samples …) — that node becomes a **leaf**.

KEY IDEA **"Greedy" learning**

At each node the tree picks the **best question right now**, without planning ahead. This is fast, but it can be short-sighted — we will see a funny example (XOR) in Chapter 7.

2.2 Measuring "how mixed is this node?"

A node is **pure** if all its rows belong to the same class (e.g. 10 "Buy" and 0 "Not Buy"). It is **maximally impure** if classes are evenly mixed. The tree needs a number for this. Two popular choices:

MATH **Gini impurity (default in scikit-learn)**

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

$\mathbf{p_k}$ = fraction of rows in the node that belong to class k; **K** = number of classes. Two classes with 50/50: G = 1 − (0.5² + 0.5²) = **0.5** (worst). A pure node: G = 1 − 1² = **0** (best).

MATH **Entropy (information-theory measure)**

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k$$

50/50 mix: H = −(0.5·log₂0.5 + 0.5·log₂0.5) = **1 bit** (worst). Pure node: H = **0**. Think of H as "how many yes/no questions do I still need to identify the class of a random row?"

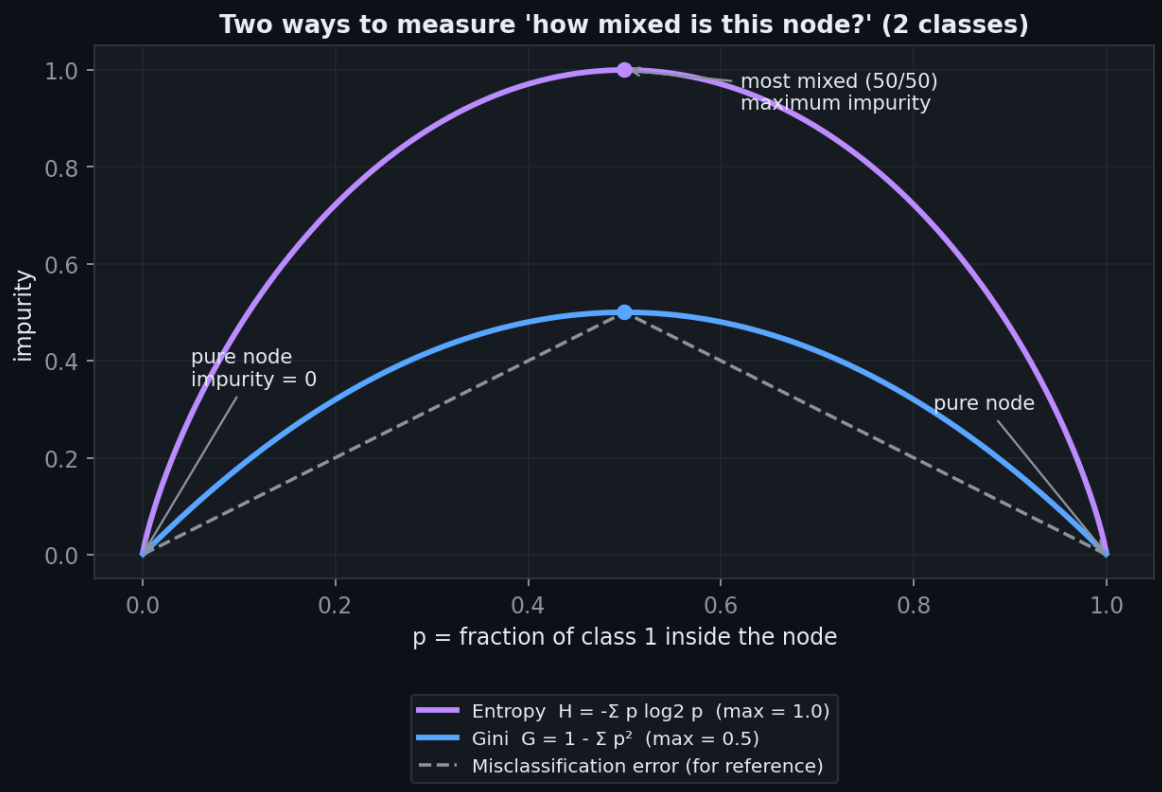


Figure 2.1 — Both measures are 0 for pure nodes and peak at a 50/50 mix. Their shapes are very similar, which is why results rarely differ much.

p (class 1 share)	0	0.1	0.3	0.5	0.9	1
Gini	0	0.18	0.42	0.50	0.18	0
Entropy (bits)	0	0.469	0.881	1.000	0.469	0

NOTE **A detail about scikit-learn**

scikit-learn computes entropy with the **natural logarithm** (not base 2). That only multiplies every value by a constant (ln 2), so it never changes *which* split wins. We use $\log_2$ here because the "bits" are easier to picture.

2.3 Scoring a split: weighted impurity and information gain

A question creates a **left** child (n_L rows) and a **right** child (n_R rows) out of a parent with n rows. A big child should matter more than a tiny one, so we **weight** each child's impurity by its size:

MATH **Weighted impurity after the split, and the gain**

$$I_{\text{split}} = \frac{n_L}{n} I(\text{left}) + \frac{n_R}{n} I(\text{right})$$

$$\text{Gain} = I(\text{parent}) - I_{\text{split}}$$

I can be Gini or Entropy. When I = Entropy the gain is called **information gain**. The tree picks the question with the **largest gain** (= the smallest weighted impurity afterwards).

2.4 A complete worked example (10 customers)

We have 10 customers described by one feature, **Age**. Label 1 = bought, 0 = did not buy.

Age	22	25	28	34	38	42	47	52	58	63
Bought?	No (0)	No (0)	No (0)	No (0)	No (0)	Yes (1)	No (0)	Yes (1)	Yes (1)	Yes (1)

Step 1 — the parent. 6 customers said No and 4 said Yes, so p = (0.6, 0.4). Gini = $1 - (0.6^2 + 0.4^2)$ = **0.48**, Entropy = **0.971** bits.

Step 2 — candidate questions. For a numeric feature the tree sorts the values and tests the **midpoint between each pair of neighbours**: (22+25)/2 = 23.5, (25+28)/2 = 26.5, … 9 candidates in total. Here is the full scan (Gini):

Question Age < t	Left (No/Yes)	Right (No/Yes)	Gini left	Gini right	Weighted Gini	Gain
23.5	1 / 0	5 / 4	0.000	0.494	0.444	0.036
26.5	2 / 0	4 / 4	0.000	0.500	0.400	0.080
31.0	3 / 0	3 / 4	0.000	0.490	0.343	0.137
36.0	4 / 0	2 / 4	0.000	0.444	0.267	0.213
40.0	5 / 0	1 / 4	0.000	0.320	0.160	0.320
44.5	5 / 1	1 / 3	0.278	0.375	0.317	0.163
49.5	6 / 1	0 / 3	0.245	0.000	0.171	0.309
55.0	6 / 2	0 / 2	0.375	0.000	0.300	0.180
60.5	6 / 3	0 / 1	0.444	0.000	0.400	0.080

Step 3 — pick the winner. The question **Age < 40** has the lowest weighted Gini (**0.16**) and therefore the biggest gain (**0.32**).

EXAMPLE **Check one row by hand: Age < 40**

Left child: 5 customers, all “No” → Gini = $1 - (1^2 + 0^2)$ = **0**.
Right child: 5 customers (1 No, 4 Yes) → Gini = $1 - (0.2^2 + 0.8^2) = 1 - 0.68$ = **0.32**.
Weighted = $(5/10)\cdot 0 + (5/10)\cdot 0.32$ = **0.16**. Gain = 0.48 − 0.16 = **0.32**.

With entropy instead: left = 0 bits; right = $-(0.2\cdot\log_2 0.2 + 0.8\cdot\log_2 0.8) = 0.722$ bits; weighted = $0.5\cdot 0 + 0.5\cdot 0.722 = 0.361$; information gain = 0.971 − 0.361 = **0.610** bits. Same winning question!

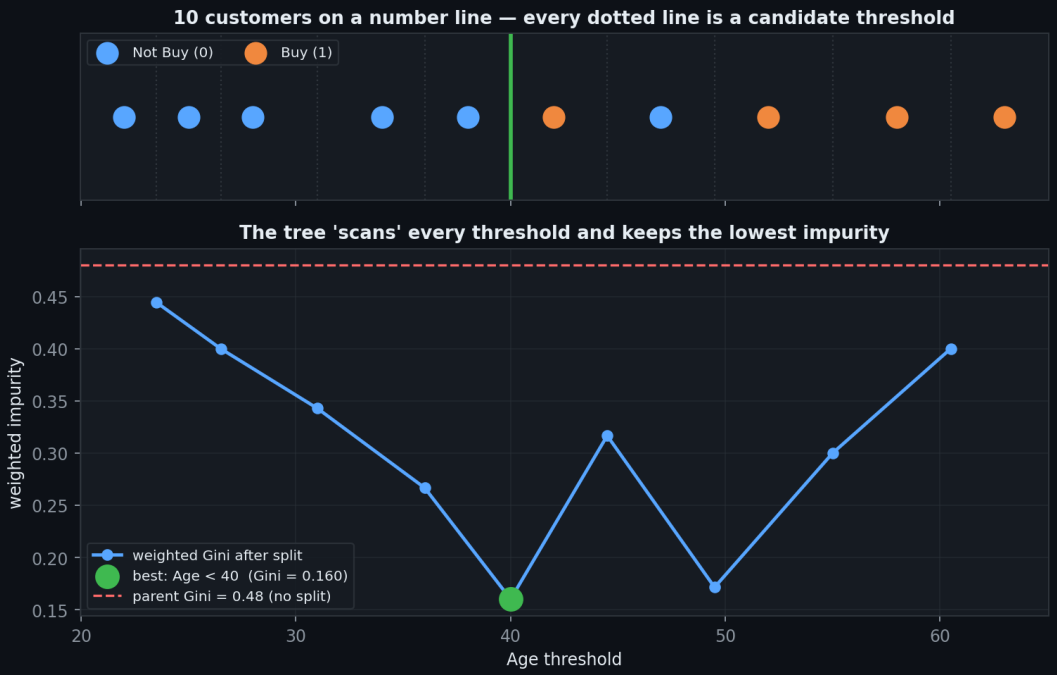


Figure 2.2 — The tree scans every candidate threshold; the green point has the lowest weighted impurity.

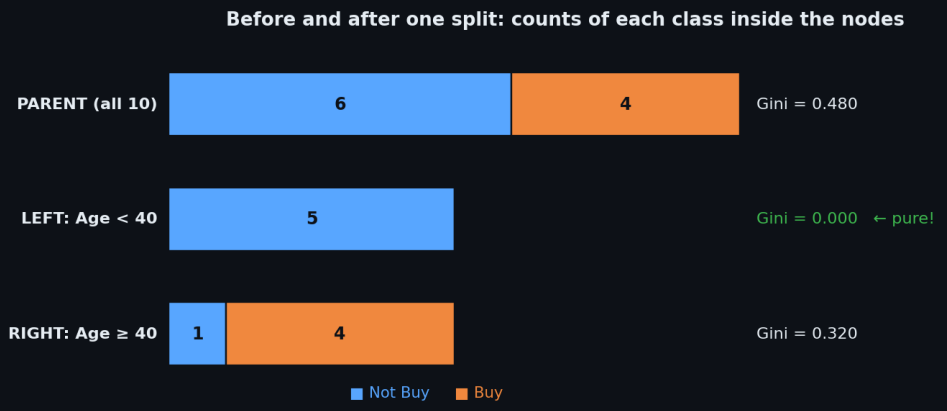


Figure 2.3 — Before and after the winning split: the left group became perfectly pure.

Step 4 — recurse. The left child is pure, so it becomes a leaf (“Not Buy”). The right child (5 customers, still mixed) repeats the whole procedure on its own rows. If we let it grow completely, scikit-learn builds a tree of depth **3** with **4** leaves:

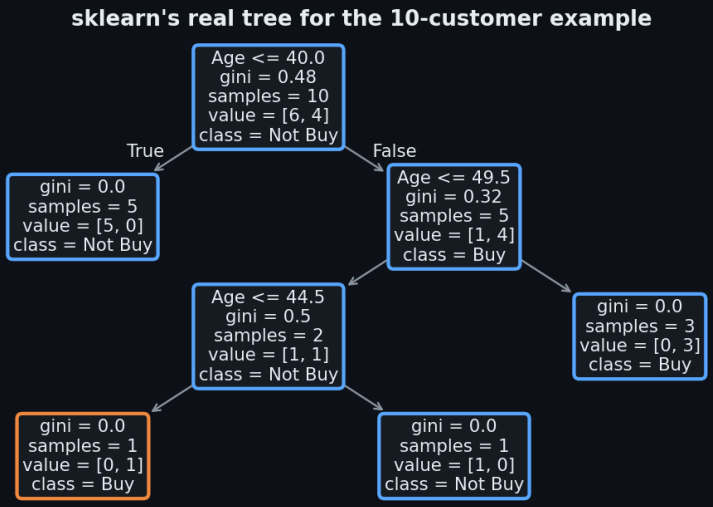


Figure 2.4 — The actual tree from scikit-learn (blue border = majority “Not Buy”, orange = majority “Buy”).

KEY IDEA **Why this matters for hyperparameters**

If we had set `max_depth=1`, the tree would stop after the first question. The right leaf would then predict "Buy" with 4/5 = 80 % confidence and be wrong for the 47-year-old. The full tree fixes that error — but with only 10 customers, those extra questions might just be **memorising noise**. Choosing where to stop is exactly what hyperparameters control.

2.5 Regression trees

For numeric targets there are no classes, so impurity is replaced by the **spread of the target values** inside a node. A leaf predicts a single number — the mean (for squared error) or the median (for absolute error) of its rows.

MATH **Regression impurity measures**

$$\text{MSE(node)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2 \qquad \hat{y}_{\text{leaf}} = \bar{y}$$

$$\text{MAE(node)} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \tilde{y}| \qquad \hat{y}_{\text{leaf}} = \tilde{y}\ \text{(median)}$$

The same weighted-impurity and gain formulas from 2.3 apply: the best split is the one that reduces the weighted MSE (or MAE) the most. MSE of a node = the **variance** of its targets.

2.6 When does a branch stop growing?

- The node is **pure** (impurity = 0) — nothing left to separate.
- A **hyperparameter limit** is hit: `max_depth`, `min_samples_split`, `min_samples_leaf` … (Chapter 4).
- No split can improve impurity (all rows identical in every feature).
- (Optional) the best gain is below `min_impurity_decrease`.

WATCH OUT **Default = no brakes**

With default settings a scikit-learn tree keeps splitting until every leaf is pure. On noisy real-world data that means **one leaf per odd row** — a perfect score on the training set and a disappointing one on new data. That problem has a name: **overfitting**.

INTERMEDIATE · CHAPTER 3

3 Overfitting, Underfitting & the Idea of Hyperparameters

Why we need knobs at all — and how to tell when they are set badly.

3.1 Parameters vs. hyperparameters

	Parameters	Hyperparameters
Who decides?	The **algorithm**, while training	**You**, before training
Examples in a tree	Which feature to ask about, the threshold, the value stored in each leaf	criterion, max_depth, min_samples_split, min_samples_leaf, max_features, ccp_alpha
How are they found?	By minimising impurity on the training data	By **trying** values and measuring on unseen data (cross-validation, Chapter 5)

Hyperparameters do not tell the tree *what* to learn; they limit *how complicated* the learned tree is allowed to become.

3.2 Overfitting and underfitting

> **ANALOGY** **The student who memorised the answers**
> Student A memorises last year's exam answers word for word (100 % on the practice paper) but fails when the questions change — **overfitting**. Student B only learned “study hard” and gets everything vaguely right but nothing precisely — **underfitting**. Student C understood the principles and scores well on new exams — **good generalisation**.

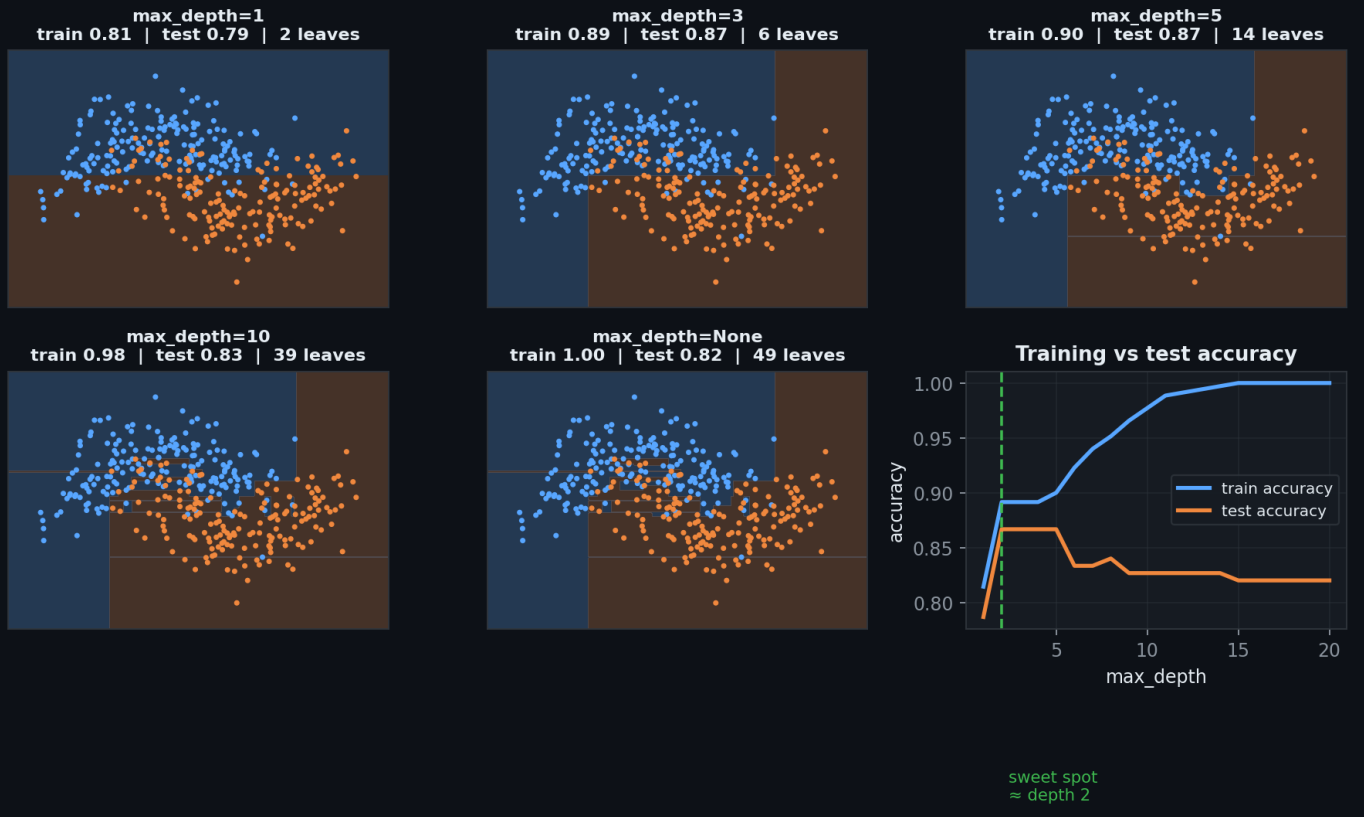


Figure 3.1 — Two-moons data with noise. Depth 1 is too simple; unlimited depth memorises noise (train 1.00, test much lower). The last panel shows the classic gap between train and test.

	Underfitting	Good fit	Overfitting
Tree looks like	very shallow, few leaves	moderate	very deep, thousands of tiny leaves
Train accuracy	low	high	**≈ 100 %**
Test accuracy	low	**high**	much lower than train
Error type	high **bias**	balanced	high **variance**
Fix	allow more depth / more leaves	—	restrict the tree (hyperparameters), prune, or use ensembles

3.3 Bias–variance trade-off (the math behind the U-curve)

MATH **Expected test error decomposes into three parts**

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Bias: error from wrong/simple assumptions (model can't capture the pattern). **Variance**: how much the model changes if trained on a different sample (it chases noise). **Noise**: randomness in the data itself — nobody can remove it. Making a tree deeper lowers bias but raises variance; the best model sits at the bottom of the total-error curve.

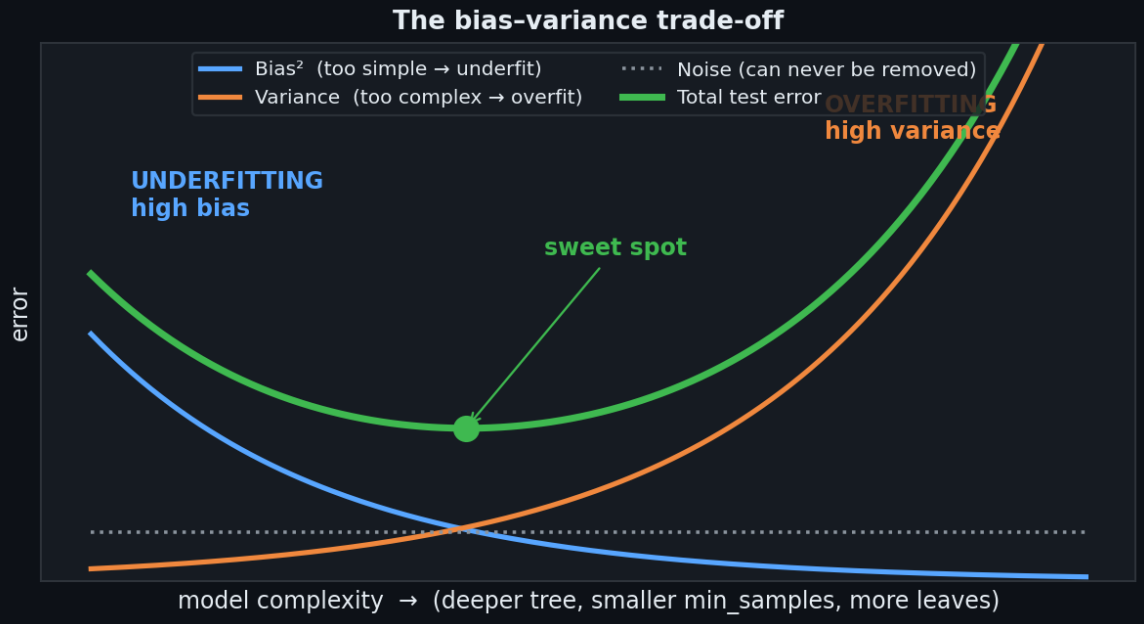


Figure 3.2 — Complexity helps until variance takes over. Your job when tuning: land near the sweet spot.

3.4 Why we hold back a test set

Training accuracy is a **flattering mirror**: a tree can always memorise it. To measure real quality we hide a portion of the data (the **test set**) and only look at it at the very end. During tuning we measure on **validation** data produced by cross-validation inside the training set (Chapter 5). Tuning on the test set would silently leak information and give over-optimistic scores (**data leakage**).

KEY IDEA **Hyperparameters = complexity dials**

Every hyperparameter in the next chapter is a dial that moves the tree toward **simpler** (more bias, less variance) or **more complex** (less bias, more variance). Tuning means finding the dial positions where test error is lowest.

INTERMEDIATE · CHAPTER 4

4 The Six Hyperparameters, One by One

What each knob does, why it helps, how it changes the model — with a picture and an example for every one.

4.1 The big picture in one table

Here is the summary from your notes, completed with the default value and the direction of each dial. “Simpler” means less risk of overfitting.

Hyperparameter	Nickname	What it controls	Default	Turn it up →
`criterion`	The Quality Metric	The math rule used to decide how to split; it measures how “pure” the resulting branches will be.	'gini'	(a choice, not a dial)
`max_depth`	The Growth Limit	Maximum number of levels (splits) from top to bottom.	None	**more complex**
`min_samples_split`	The Splitting Threshold	Minimum data points a node must have before it may split again.	2	**simpler**
`min_samples_leaf`	The Endpoint Rule	Minimum data points that must exist in a final leaf.	1	**simpler**
`max_features`	The Feature Lottery	Maximum number of features (columns) examined when looking for the best split at a node.	None (all)	closer to a plain greedy tree
`ccp_alpha`	The Pruning Shear	Complexity penalty used for pruning after the tree has grown.	0.0	**simpler**

4.2 `criterion` — The Quality Metric

How it works	This is the mathematical rule the tree uses to decide how to split the data. It measures how “pure” the resulting branches will be — every candidate question is scored with it (Section 2.3) and the best-scoring one wins.
Classification	**Gini impurity** (default) or **Entropy** (information gain).
Regression	**Squared error** (MSE) or **Absolute error** (MAE).
Grid values	Classification: `['gini', 'entropy']` Regression: `['squared_error', 'absolute_error']`

MATH **The four formulas side by side**

$$\text{Gini} = 1 - \sum_k p_k^2 \qquad \text{Entropy} = -\sum_k p_k \log_2 p_k$$

$$\text{MSE} = \frac{1}{n}\sum_i (y_i - \bar{y})^2 \qquad \text{MAE} = \frac{1}{n}\sum_i |y_i - \tilde{y}|$$

Gini vs. Entropy — which one?

- **Gini** is slightly faster (no logarithm) and tends to isolate the most frequent class in its own branch.
- **Entropy** penalises mixed nodes a bit more strongly and can give slightly more balanced trees.
- In practice the two give **very similar accuracy** (compare the curves in Figure 2.1), so treat the choice as a cheap thing to include in the grid, not as the main lever.

Squared error vs. Absolute error (regression)

MSE squares the mistakes, so a single huge outlier counts enormously and drags the leaf's average toward itself. MAE uses the **median**, which barely notices outliers — but it is slower to compute. In Figure 4.1 the red crosses are outliers: the blue (MSE) tree spends its splits chasing them (the narrow spikes and dips), while the green (MAE) tree mostly ignores them and follows the true curve.

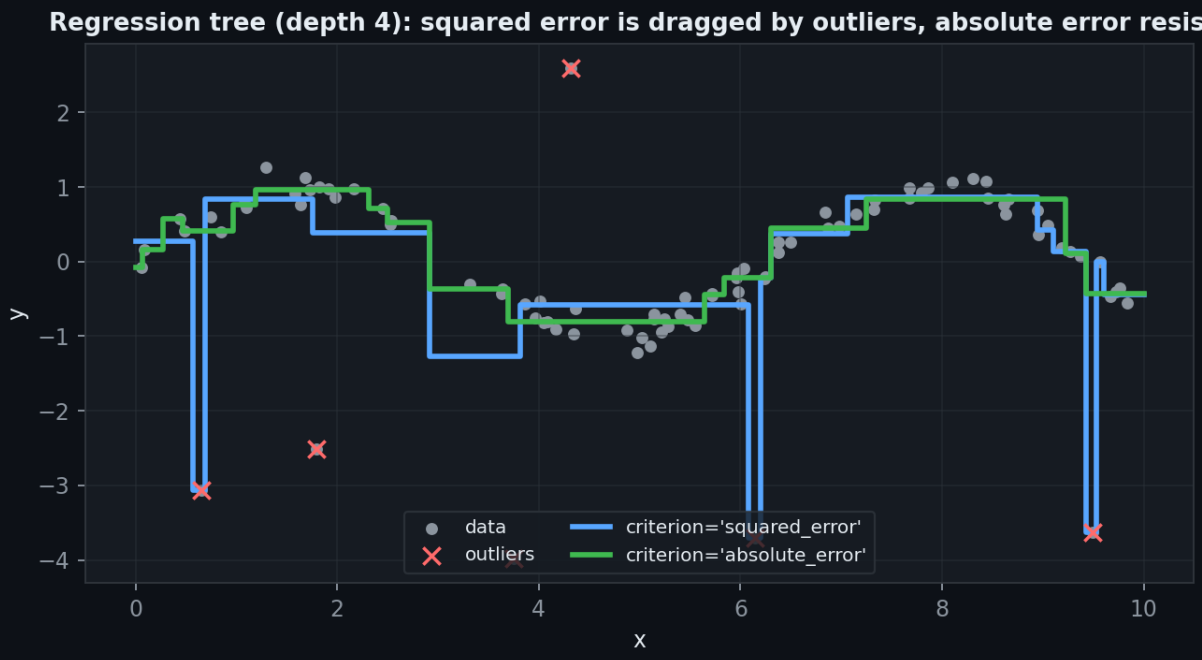


Figure 4.1 — Regression tree, depth 4: criterion changes how outliers are treated.

4.3 max_depth — The Growth Limit

How it works	Sets the absolute limit on how many splits (levels) the tree can make from the root node down to the furthest leaf. If set to **None**, the tree continues splitting until every leaf is mathematically pure or contains fewer samples than *min_samples_split*.
How it helps	It is the **primary stopping mechanism** that prevents the tree from memorising the training dataset.
How it affects the model	A very high or unrestricted depth leads to **severe overfitting (high variance)**: the tree creates hyper-specific rules for outliers and noise. A very low depth leads to **underfitting (high bias)**: the tree cannot capture the underlying patterns in the data.
Grid values	`[None, 3, 5, 10, 15]` (detailed notes) or `[None, 5, 10, 15, 20]` (summary table). Including **None** provides a baseline unconstrained tree, while the integers test various restriction levels to identify the optimal cut-off.

MATH **How big can a tree get?**

$$\text{leaves}_{\max} = 2^{\text{max_depth}}$$

Depth 3 → at most 8 leaves, depth 5 → 32, depth 10 → 1 024, depth 15 → 32 768. In reality the number of training rows caps the leaves much earlier — but you can see why unlimited depth is dangerous: capacity doubles with every extra level.

EXAMPLE **What we saw in Figure 3.1**

Depth 1: both train and test accuracy are poor (underfit). Unlimited depth: train accuracy 1.00 but test accuracy clearly lower (overfit). On that data the test accuracy peaked at about **depth 2** — a modest depth.

4.4 min_samples_split — The Splitting Threshold

How it works	Defines the exact **minimum number of data points a parent node must contain to be eligible for another split**. For example, a value of 10 means any node with 9 or fewer samples becomes a terminal leaf automatically.
How it helps	It forces the algorithm to stop splitting early when a node does not have enough statistical evidence to justify further division.
How it affects the model	Lower values (like the default of 2) allow the tree to create highly customised branches — a node with just 2 data points can split, leading to very specific, overfitted rules. Higher values enforce broader, more generalised rules, reducing variance but potentially increasing bias if the threshold is set too high.
Grid values	`[2, 5, 10, 20]`

4.5 min_samples_leaf — The Endpoint Rule

How it works	Dictates the absolute **minimum number of samples that must remain in a leaf** (terminal node) after a split is executed. If a proposed split would result in a left or right leaf with fewer samples than this threshold, the split is aborted.
How it helps	Unlike *min_samples_split* (which restricts the parent), this ensures the final resulting buckets have a meaningful amount of data. It is highly effective at **smoothing** the model, particularly for regression tasks.
How it affects the model	Setting this to 1 allows the tree to isolate single outliers (a leaf can represent a single outlier). Increasing the value forces the tree to group data points together, immediately reducing overfitting and making predictions more robust to noise.
Grid values	`[1, 5, 10, 20]` (detailed notes) or `[1, 2, 5, 10]` (summary table).

min_samples_split vs. min_samples_leaf — the difference everyone mixes up

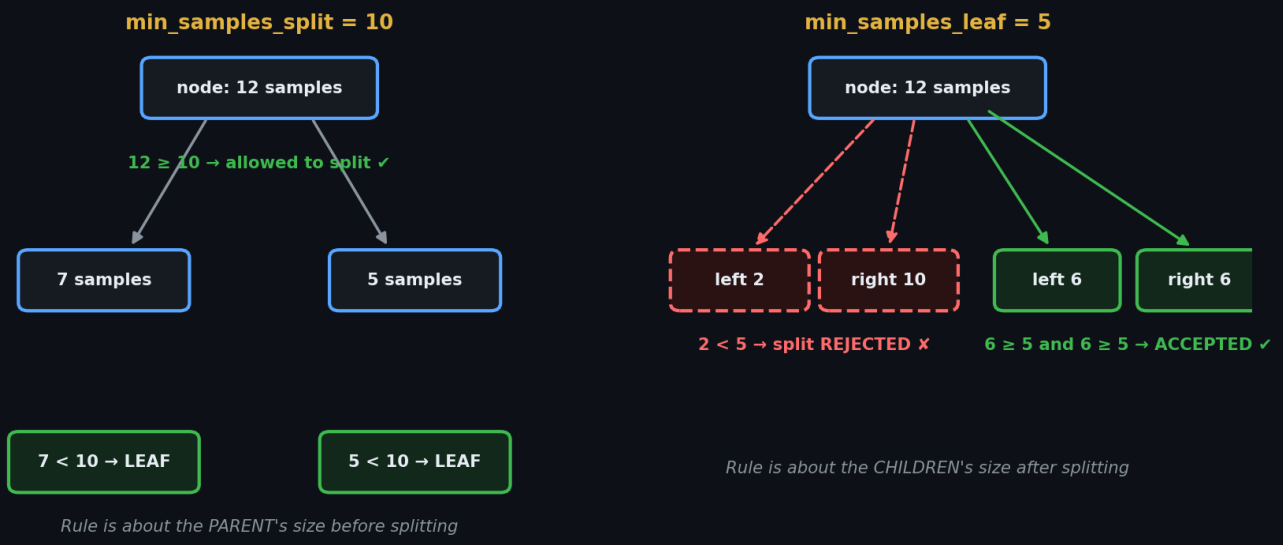


Figure 4.2 — One rule looks at the node before splitting, the other at the pieces after splitting.

	`min_samples_split`	`min_samples_leaf`
Looks at	size of the **parent** before the split	size of **each child** after the split
If violated	node is not split at all (becomes a leaf)	that particular split is rejected (another split may still be tried)
Typical effect	stops growth early	guarantees every leaf has real evidence; smooths predictions
Rule of thumb	min_samples_split ≥ 2 × min_samples_leaf (otherwise leaf rule dominates)	often the more effective of the two

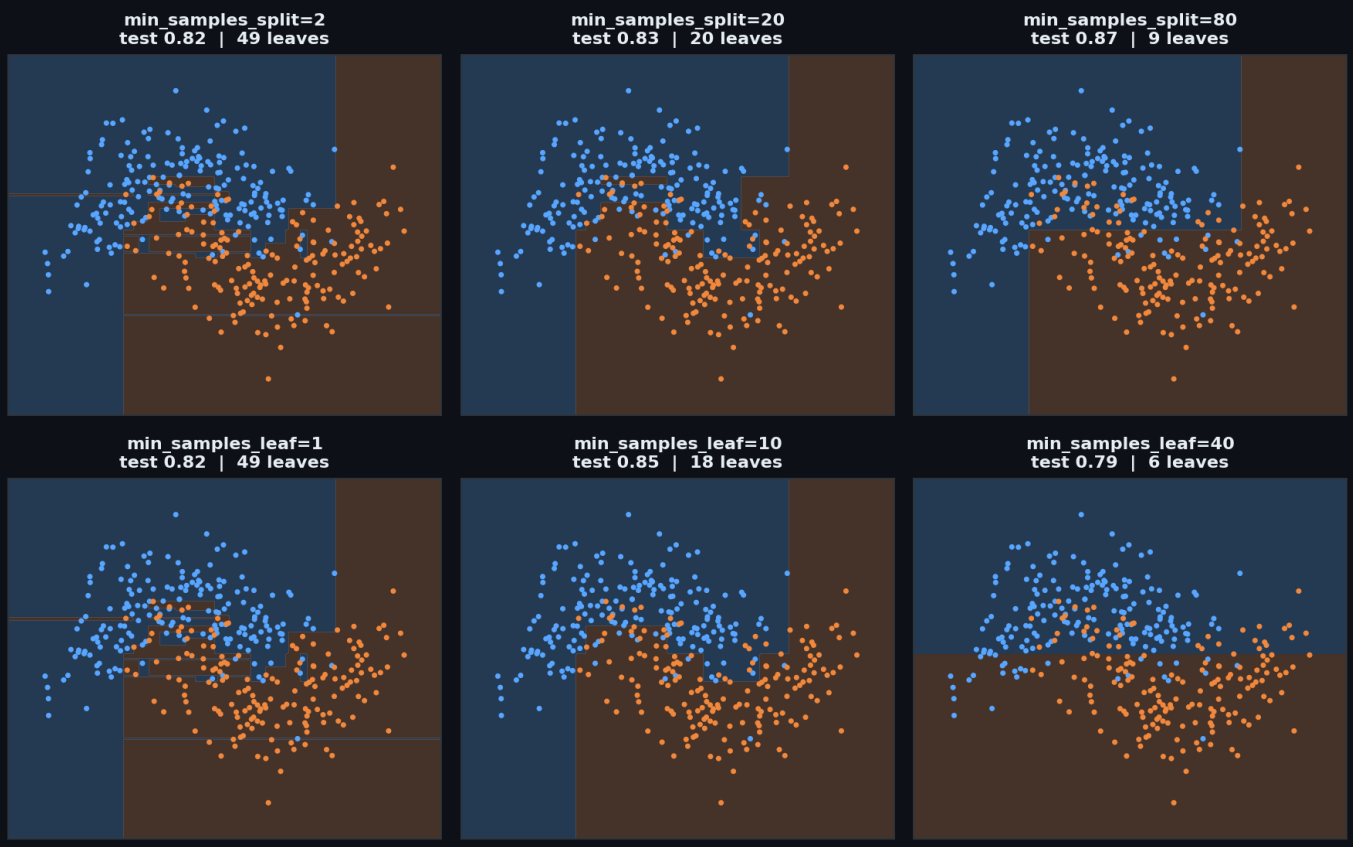


Figure 4.3 — Top row: min_samples_split = 2, 20, 80. Bottom row: min_samples_leaf = 1, 10, 40. Larger values → smoother boundary, fewer leaves.

min_samples_split	2	5	10	20	40	80
leaves	49	44	29	20	12	9
test accuracy	0.820	0.840	0.827	0.833	0.853	0.867

min_samples_leaf	1	2	5	10	20	40
leaves	49	41	26	18	11	6
test accuracy	0.820	0.813	0.847	0.853	0.853	0.787

4.6 max_features — The Feature Lottery

How it works	Determines the **maximum number of distinct features the algorithm is permitted to evaluate** when searching for the absolute best split at any given node. Instead of looking at all 100 features every time it splits, setting this to **sqrt** makes it look at just 10 random features.
How it helps	Instead of exhaustively scanning every single variable, it introduces **randomness** into the splitting process. This reduces computational cost and prevents the tree from becoming overly reliant on one dominant feature.
How it affects the model	Evaluating all features (the default) can cause the tree to greedily select the same dominant features at the top of the hierarchy, missing nuanced patterns. Restricting this limits overfitting by forcing the tree to build different structures, which is particularly beneficial when multiple features are highly correlated.
Grid values	`[None, 'sqrt', 'log2']`

MATH How many features is that?

$$\texttt{sqrt}: m = \lfloor \sqrt{d} \rfloor \qquad \texttt{log2}: m = \lfloor \log_2 d \rfloor \qquad \text{None}: m = d$$

d = total number of features. Example with d = 100: sqrt → **10** features per split, log2 → **6**, None → **100**. For the 30-feature breast-cancer data: sqrt → 5, log2 → 4.

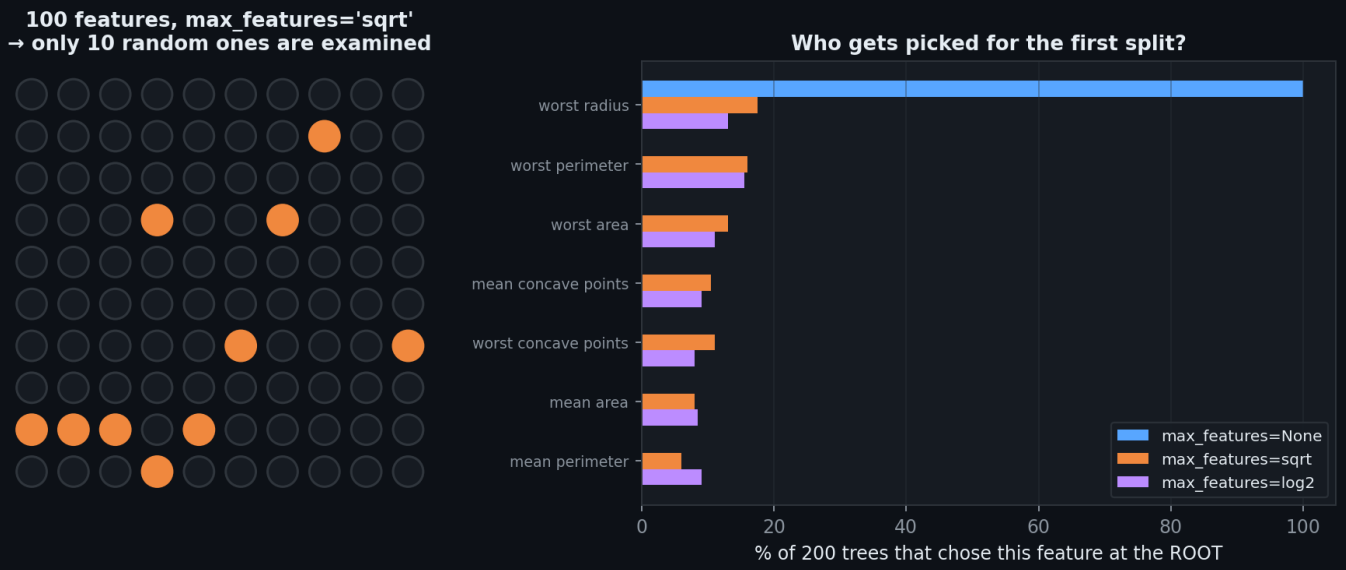


Figure 4.4 — Left: the lottery picks 10 of 100 features. Right: over 200 trees, with max_features=None the same feature always wins the root; with sqrt/log2 the root varies — more diverse trees.

EXAMPLE **Measured on the breast-cancer data**

Over 200 trees trained on the same data: **max_features=None** picked 1 distinct feature at the root (every single time); **'sqrt'** picked 16 different ones; **'log2'** picked 19. That diversity is exactly what Random Forests exploit (Chapter 7).

4.7 ccp_alpha — The Pruning Shear

How it works	The **complexity parameter used for pruning** (Minimal Cost-Complexity Pruning). The tree is first grown large; then branches are snipped off, weakest first, based on a cost-penalty formula.
How it helps	It **penalises the tree for having too many nodes**. A higher alpha removes weaker branches after the tree has grown, leaving only the most important splits.
Grid values	`[0.0, 0.01, 0.1]` (optional). 0.0 means "no pruning".

MATH **The cost-complexity formula**

$$R_\alpha(T) = R(T) + \alpha\,|\tilde{T}|$$

$$\alpha_{\text{eff}}(t) = \frac{R(t) - R(T_t)}{|\tilde{T}_t| - 1}$$

R(T) = total impurity of the leaves of tree T (the "error"). **|T|** = number of leaves (the "size"). **α** = price you pay for each leaf. For an internal node t, **R(t)** is its impurity if it were collapsed into a single leaf and $\mathbf{T_t}$ is the branch below it; α_{eff} says "how much extra error per leaf saved do I accept by cutting this branch?". Branches with the **smallest** α_{eff} are the weakest links and are cut first, whenever $\alpha_{\text{eff}} \le$ ccp_alpha.

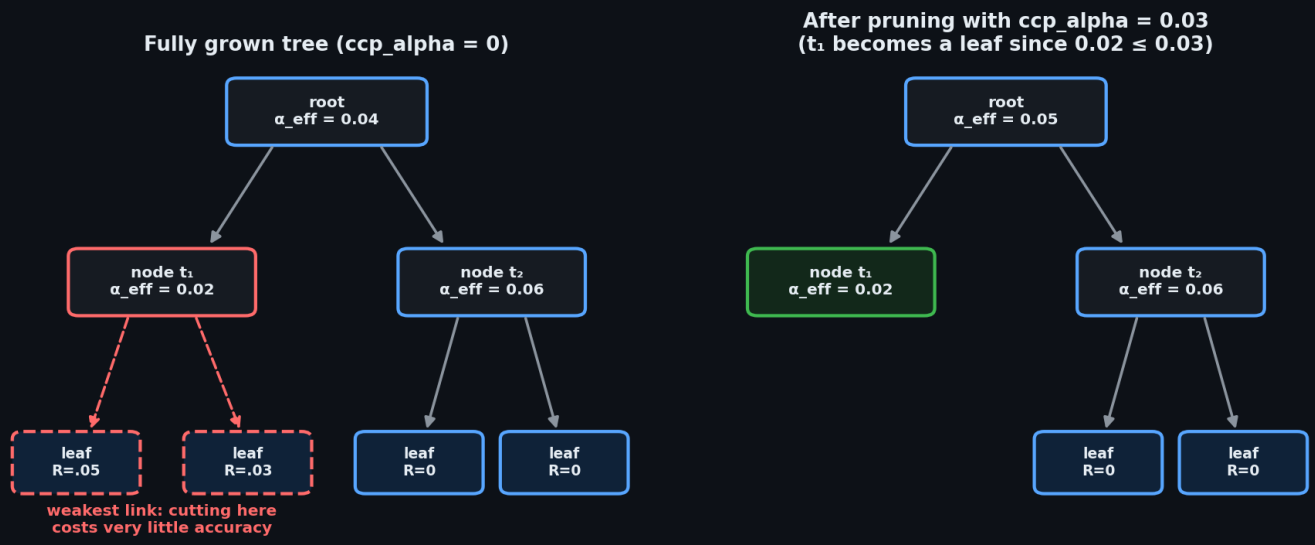


Figure 4.5 — Pruning in action. Node t_1 has the lowest α_eff (0.02), so it is cut first; after that the root's α_eff rises to (0.20−0.10)/(3−1)=0.05.

EXAMPLE **The numbers in Figure 4.5**

t_1: $R(t_1)$=0.10, its two leaves total 0.05+0.03=0.08 → α_eff = (0.10−0.08)/(2−1) = **0.02**.
t_2: $R(t_2)$=0.06, its leaves total 0 → α_eff = (0.06−0)/(2−1) = **0.06**.
Root: R=0.20, whole tree R(T)=0.08 with 4 leaves → α_eff = (0.20−0.08)/(4−1) = **0.04**.
With ccp_alpha = 0.03 only t_1 qualifies (0.02 ≤ 0.03) → it becomes a leaf with R=0.10. Now the root's α_eff is (0.20−0.10)/(3−1) = 0.05 > 0.03, so pruning stops.

```
tree = DecisionTreeClassifier(random_state=0)
path = tree.cost_complexity_pruning_path(X_train, y_train)   # all 'natural' alpha values
print(path.ccp_alphas)                                       # try THESE values in your grid
```



Figure 4.6 — Pruning path on the two-moons data: the tree shrinks from 49 leaves; the best test accuracy appears at α≈0.0092 with only 7 leaves.

PRO TIP **Use ccp_alpha instead of endlessly guessing**

Instead of endlessly guessing combinations of **max_depth** and **min_samples_***, use **ccp_alpha**. This technique grows a large tree and then mathematically snips off the weakest branches based on a cost-penalty formula. It is often the fastest route to a perfectly sized, generalised tree.

Alpha is measured in impurity units, so a good value depends on your dataset. Rather than guessing 0.01 or 0.1, take values from `cost_complexity_pruning_path` (above).

4.8 Other parameters worth knowing (extras)

Parameter	What it does	When to use
`class_weight`	Gives rare classes a bigger weight in the impurity calculation (e.g. 'balanced').	Imbalanced classification (Chapter 7)
`max_leaf_nodes`	Directly caps the number of leaves; grows best-first.	When you want a tree with at most N rules
`min_impurity_decrease`	Only split if the (weighted) impurity drops by at least this amount.	An alternative way to stop useless splits
`splitter`	'best' checks every threshold; 'random' tests random ones.	Rarely; adds randomness / speed
`random_state`	Seed for tie-breaking and feature sampling.	**Always** set for reproducible results

PRO TIP **Which dials first?**

A practical order: (1) **min_samples_leaf** and **max_depth** (or **ccp_alpha**) — they control complexity the most; (2) **max_features**; (3) **min_samples_split**; (4) **criterion** last, since Gini vs entropy rarely matters. The grid search in the next chapter simply tries them all together.

ADVANCED · CHAPTER 5

5 Grid Search: Finding the Best Settings Automatically

Recommended search values, how cross-validation scores them, and how many fits you are really asking for.

5.1 Recommended grid-search values (your table)

Hyperparameter	Preferred values (Classification)	Preferred values (Regression)
`criterion`	`['gini', 'entropy']`	`['squared_error', 'absolute_error']`
`max_depth`	`[None, 5, 10, 15, 20]`	`[None, 5, 10, 15, 20]`
`min_samples_split`	`[2, 5, 10, 20]`	`[2, 5, 10, 20]`
`min_samples_leaf`	`[1, 2, 5, 10]`	`[1, 2, 5, 10]`
`max_features`	`[None, 'sqrt', 'log2']`	`[None, 'sqrt', 'log2']`
`ccp_alpha`	`[0.0, 0.01, 0.1]` (Optional)	`[0.0, 0.01, 0.1]` (Optional)

PRO TIP **Implementation note (from your notes)**

When running **GridSearchCV** in Python (scikit-learn), using `max_depth: [None]` is important because it establishes your **baseline unconstrained tree**, while the integer values test whether restricting the depth improves performance on unseen data.

Other value lists that appear in the detailed notes are equally valid starting points: `max_depth [None, 3, 5, 10, 15]` and `min_samples_leaf [1, 5, 10, 20]`. The complete example in Chapter 6 uses `max_depth [None, 5, 10, 15]` and leaves out ccp_alpha to keep the search small.

5.2 What is a grid search?

ANALOGY **Trying every combination on a menu**

You want the perfect pizza: 2 doughs × 3 sauces × 4 toppings = 24 pizzas. A grid search simply **bakes and tastes all 24**, then tells you which was best. For trees, “pizza” = a hyperparameter combination and “tasting” = measuring accuracy on data the tree did not train on.

5.3 How each combination is scored: cross-validation

Splitting once into train/validation could be lucky or unlucky. **k-fold cross-validation** splits the training data into k equal folds; each fold takes one turn as the validation set while the other k−1 folds train the tree. The k scores are averaged.



Figure 5.1 — 5-fold cross-validation (cv=5).

MATH **Cross-validated score and the number of fits**

$$\text{CV score} = \frac{1}{k}\sum_{j=1}^{k} s_j$$

$$\text{total fits} = \left(\prod_i n_i\right) \times k$$

n_i = number of values you try for hyperparameter i; **k** = number of folds. The product grows **multiplicatively** — adding one more hyperparameter with 4 values multiplies the work by 4.

Grid	Combinations	× 5 folds = fits
Chapter 6 grid: 2 × 4 × 4 × 4 × 3	**384**	**1 920** (verbose output says exactly this)
Table above without ccp_alpha: 2 × 5 × 4 × 4 × 3	480	2 400
Table above with ccp_alpha: 2 × 5 × 4 × 4 × 3 × 3	1 440	7 200

This is why `n_jobs=-1` (use every CPU core) matters: the fits are independent, so they can run in parallel.

5.4 Reading the result: a heat-map of the search

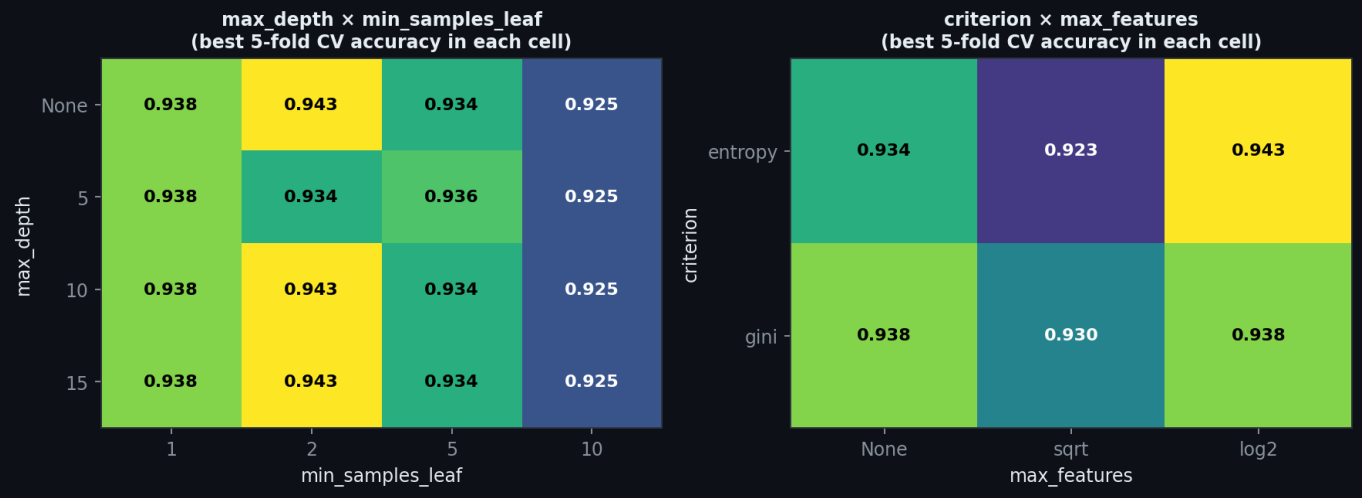


Figure 5.2 — Real results of the Chapter 6 search on the breast-cancer data: best 5-fold CV accuracy for pairs of hyperparameters.

Across all 384 combinations the mean CV accuracy ranged from **0.892** to **0.943**. Notice how **flat** the map is near the top: many settings are practically tied. That is common, and the reason why a tuned tree is a small improvement rather than a miracle.

> **PRO TIP** **Handy ways to inspect a search**
> `pd.DataFrame(grid_search.cv_results_)` gives every combination with its mean score and standard deviation; sort by `rank_test_score`. If several settings are within one standard deviation of the best, prefer the **simplest** (smaller tree).

5.5 When the grid gets too big

- **Coarse → fine:** run a wide, cheap grid first, then a narrow grid around the winner.
- **RandomizedSearchCV:** samples N random combinations; usually finds a nearly-as-good result in a fraction of the time.
- **Use ccp_alpha** from `cost_complexity_pruning_path` instead of many depth/sample combos (Section 4.7).
- **Parallelise:** `n_jobs=-1`.

ADVANCED · CHAPTER 6

6 The Complete Code, Line by Line

A full, runnable grid search for a decision tree — exactly as in your notes — with every step explained and the real output.

6.1 The data and the goal

We use scikit-learn's built-in **breast-cancer dataset**: 569 patients, 30 numeric measurements of cell nuclei, and a label (0 = malignant, 1 = benign). Goal: find the decision-tree settings that classify new patients best. The data is split 80 % / 20 % into 455 training rows and 114 test rows.

6.2 The full program

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load and prepare your dataset
# Replace this with your actual data loading logic (e.g., pd.read_csv('data.csv'))
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# 2. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Initialize the base model
# random_state ensures reproducibility across different runs
dt_model = DecisionTreeClassifier(random_state=42)

# 4. Define the hyperparameter grid
# These values target the most impactful parameters for Decision Trees
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 5, 10],
    'max_features': [None, 'sqrt', 'log2']
}

# 5. Configure GridSearchCV
# cv=5: 5-fold cross-validation
# n_jobs=-1: Utilize all available CPU cores for speed
# scoring='accuracy': The metric used to evaluate the best model
grid_search = GridSearchCV(
    estimator=dt_model,
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    scoring='accuracy',
    verbose=1 # Prints progress updates to the console
)

# 6. Execute the grid search on the training data
grid_search.fit(X_train, y_train)

# 7. Extract the best model and its parameters
best_tree = grid_search.best_estimator_
print(f"Best Hyperparameters: {grid_search.best_params_}")
print(f"Best Cross-Validation Accuracy: {grid_search.best_score_:.4f}")

# 8. Evaluate the optimized model on unseen test data
y_pred = best_tree.predict(X_test)
test_accuracy = accuracy_score(y_test, y_pred)

print(f"\nTest Set Accuracy: {test_accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

6.3 What each step does

Step	Code	What happens and why
1	`load_breast_cancer()`	Loads the data and wraps it in a pandas DataFrame **X** (features) and Series **y** (labels). Replace with `pd.read_csv()` for your own data.
2	`train_test_split(...)`	Hides 20 % of the rows as the **test set**. Everything else (training set) is used for learning *and* tuning.
3	`DecisionTreeClassifier(random_state=42)`	A base tree with default (unrestricted) settings. GridSearchCV will clone it once per combination.
4	`param_grid = {...}`	A dictionary: hyperparameter name → list of values to try. 2 × 4 × 4 × 4 × 3 = **384** combinations.
5	`GridSearchCV(...)`	Wraps the model and the grid. **cv=5**: 5-fold cross-validation; **n_jobs=-1**: all CPU cores; **scoring='accuracy'**: the number used to rank combinations; **verbose=1**: print progress.
6	`grid_search.fit(X_train, y_train)`	The expensive part: 384 × 5 = **1 920 trees** are trained and validated. Afterwards (default `refit=True`) the best combination is re-trained on the *whole* training set.
7	`best_estimator_ / best_params_ / best_score_`	The winning, already re-trained tree; its settings; and its average cross-validation accuracy.
8	`predict + accuracy_score + classification_report`	**First and only** use of the test set: an honest estimate of how the tuned tree behaves on new patients.

6.4 Four lines worth understanding deeply

PRO TIP **stratify=y**

Ensures the train and test sets have the **exact same proportion of target classes**, preventing imbalanced splits from ruining the evaluation. (Here: ≈37 % malignant in both.)

PRO TIP **random_state=42**

Locks the random number generator so your tree builds the **exact same way every time** you run the script, which is mandatory for reliable debugging and tuning. (42 is just a convention — any fixed number works.)

PRO TIP **n_jobs=-1**

Commands the machine to run the cross-validation tests **in parallel across all CPU cores**, significantly cutting down training time.

PRO TIP **best_estimator_**

Automatically saves and isolates the **highest-performing model** from the grid search so you can immediately run predictions on your test set **without retraining**.

6.5 The real output

This is what the program printed when it was run for this tutorial (scikit-learn 1.8):

```
Fitting 5 folds for each of 384 candidates, totalling 1920 fits
Best Hyperparameters: {'criterion': 'entropy', 'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_
Best Cross-Validation Accuracy: 0.9429

Test Set Accuracy: 0.9561
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        42
           1       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

6.6 How to read it

Best hyperparameters. The winning tree uses entropy, unlimited depth, `log2` features per split, `min_samples_leaf=2` and `min_samples_split=10`. Its final depth is 7 with 18 leaves.

CV accuracy vs. test accuracy. 0.9429 (average of 5 validation folds) and 0.9561 (test set) are close — a good sign that the tuning did not overfit the validation data. For comparison, the default, untuned tree scores 1.00 on its training data but only **0.9123** on the test set.

WATCH OUT **Do not over-trust small differences**

The test set has only 114 rows, so **one patient = 0.88 percentage points**. The gap between the default tree (0.912) and the tuned tree (0.956) is about 5 patient(s). Many hyperparameter combinations are practically tied (Figure 5.2), and with another scikit-learn version, CPU or seed you may see a different "best" combination with the same score. Judge tuning with **repeated cross-validation**, not a single small test set.

MATH **What the classification report numbers mean**

$$\text{Precision} = \frac{TP}{TP+FP} \qquad \text{Recall} = \frac{TP}{TP+FN} \qquad F_1 = \frac{2 \cdot P \cdot R}{P+R} \qquad \text{Accuracy} = \frac{TP+TN}{N}$$

Precision: of the patients predicted as a class, how many truly were. **Recall**: of the patients who truly were that class, how many did we find. **F1**: their harmonic mean. **Support**: number of test rows of that class.

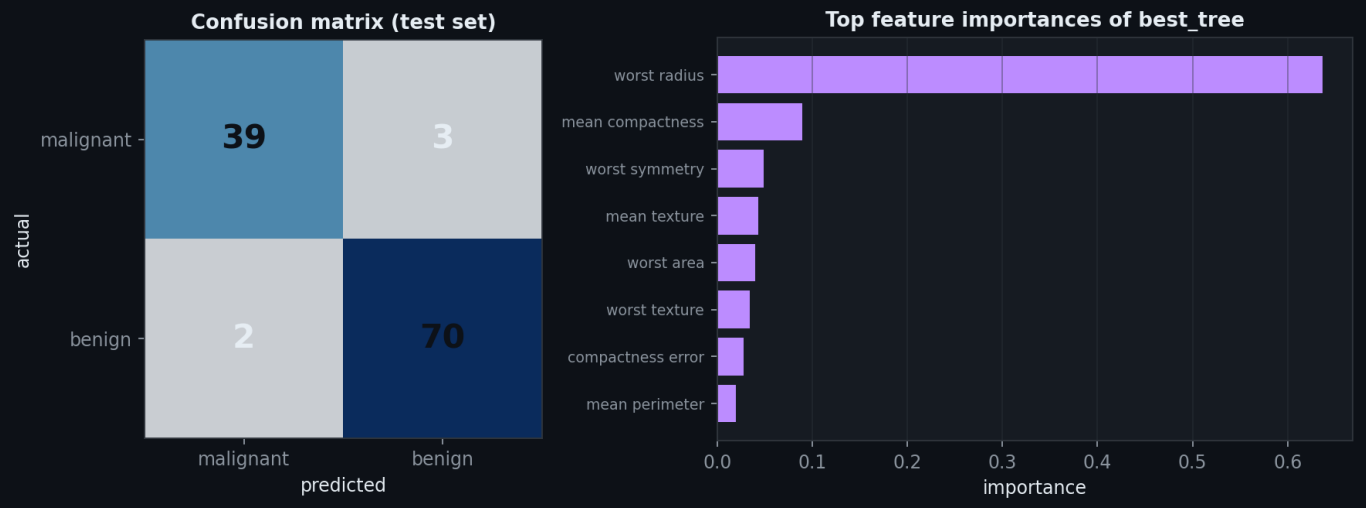


Figure 6.1 — Left: confusion matrix. Of 42 truly malignant patients 39 were found and 3 were missed; of 72 benign patients 70 were correct and 2 were flagged wrongly. Right: the most important features.

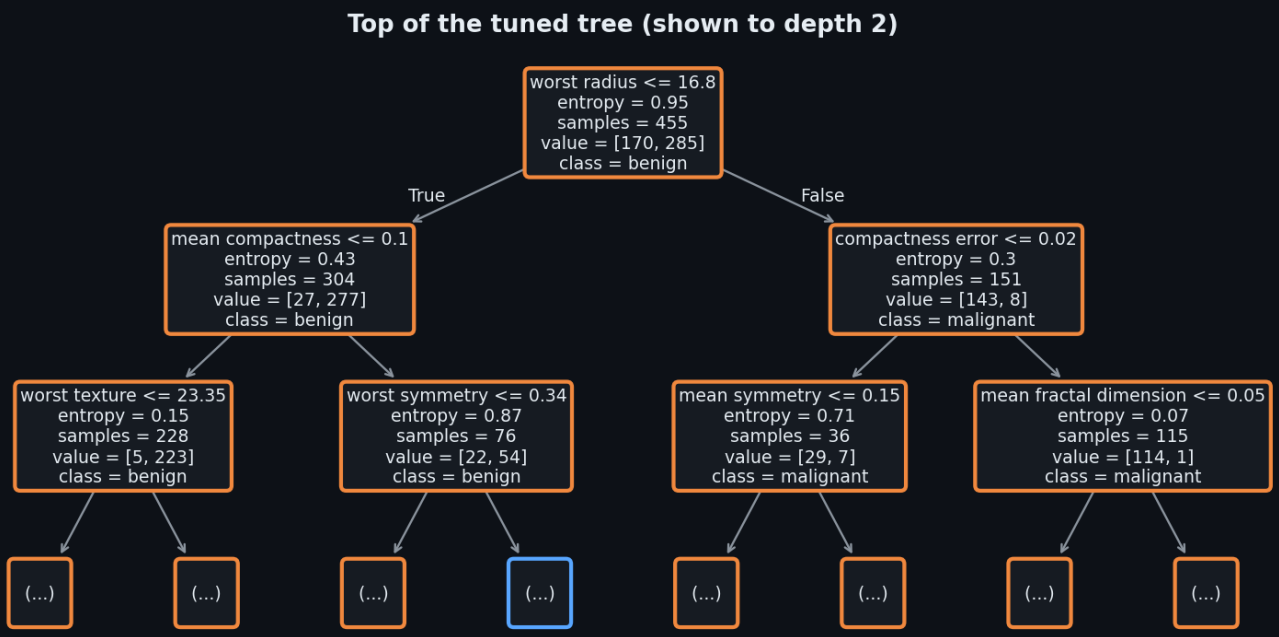


Figure 6.2 — The top two levels of the tuned tree. Each box shows the question, impurity, number of samples, class counts and majority class.

Rank	Feature	Importance
1	worst radius	0.636
2	mean compactness	0.089
3	worst symmetry	0.049
4	mean texture	0.044
5	worst area	0.040

PRO · CHAPTER 7

7 Pro Tips & Best Practices

Everything your notes list as advice — each tip in technical form and in plain words, with a picture or a measured experiment.

7.1 Use cost-complexity pruning (ccp_alpha)

Technical. Instead of endlessly guessing combinations of `max_depth` and `min_samples_*`, use `ccp_alpha`. This technique grows a large tree and then mathematically snips off the weakest branches based on a cost-penalty formula. It is often the fastest route to a perfectly sized, generalised tree.

IN PLAIN WORDS **Like pruning a rose bush**

Let the bush grow wild first, then cut the weak, useless twigs. You do not need to guess in advance how tall the bush should be. See Section 4.7 for the formula, Figure 4.5 for the picture and the two-line recipe using `cost_complexity_pruning_path`.

7.2 Skip feature scaling — and create smart data instead

Technical. Decision trees split data based on strict thresholds (e.g., *Age* > *30*), meaning they are completely **invariant to the scale** of your variables. Do not waste computational resources or pipeline complexity applying Standard Scalers or Min-Max Scalers.

MATH **Why scaling cannot change a tree**

$$x < t \iff \frac{x-\mu}{\sigma} < \frac{t-\mu}{\sigma}$$

Scaling is a **monotonic** transformation: it keeps the order of the values. The tree only cares about *which rows fall on which side* of a threshold, and that does not change. Experiment: we trained one tree on raw data and one on standardised data — predictions identical on every row: **True**.

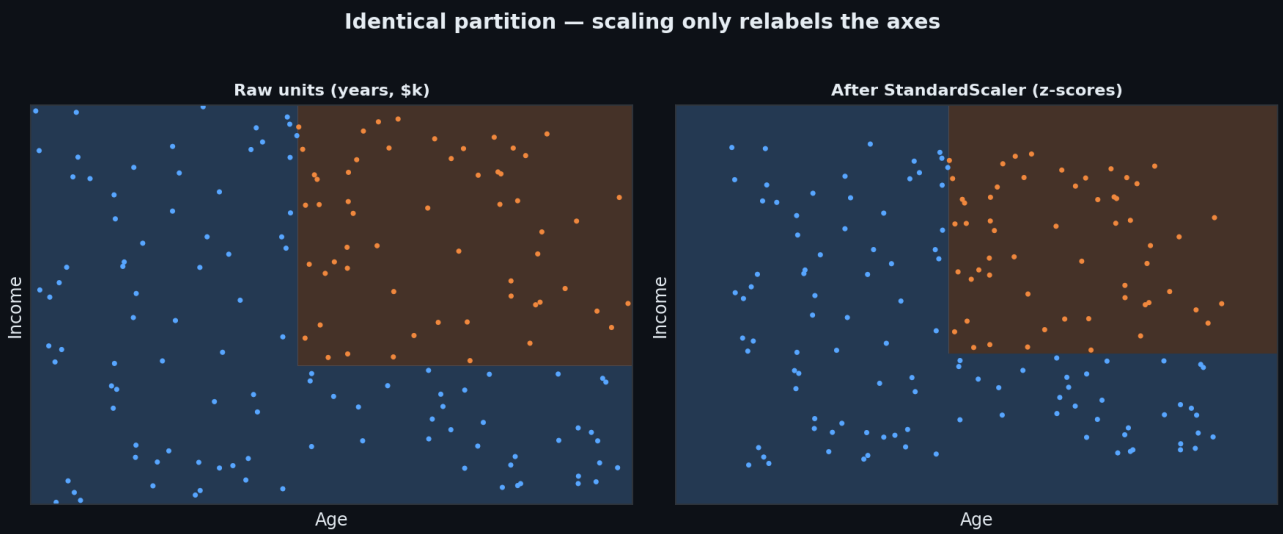


Figure 7.1 — Same decision regions in raw units and in z-scores.

WATCH OUT **Scaling still matters for other models**

k-NN, SVM, logistic regression, neural networks and PCA **do** need scaling. Trees (and Random Forests, Gradient Boosting) do not.

In plain words — “Focus on creating smart data, not resizing it”. Decision trees do not care if your numbers are big (like measuring in miles) or small (like measuring in inches). Changing the scale of your numbers is a waste of time. Instead, help the tree by creating **smarter data**. For example, if you have a “Height” column and a “Weight” column, doing the math yourself to create a new “BMI” column gives the tree a much clearer clue to work with.

MATH Feature engineering example

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height (m)}^2}$$

A tree can only draw **straight, axis-parallel** lines. The real rule “BMI ≥ 27” is a curve in the height–weight plane, so the tree needs a staircase of many splits to imitate it.

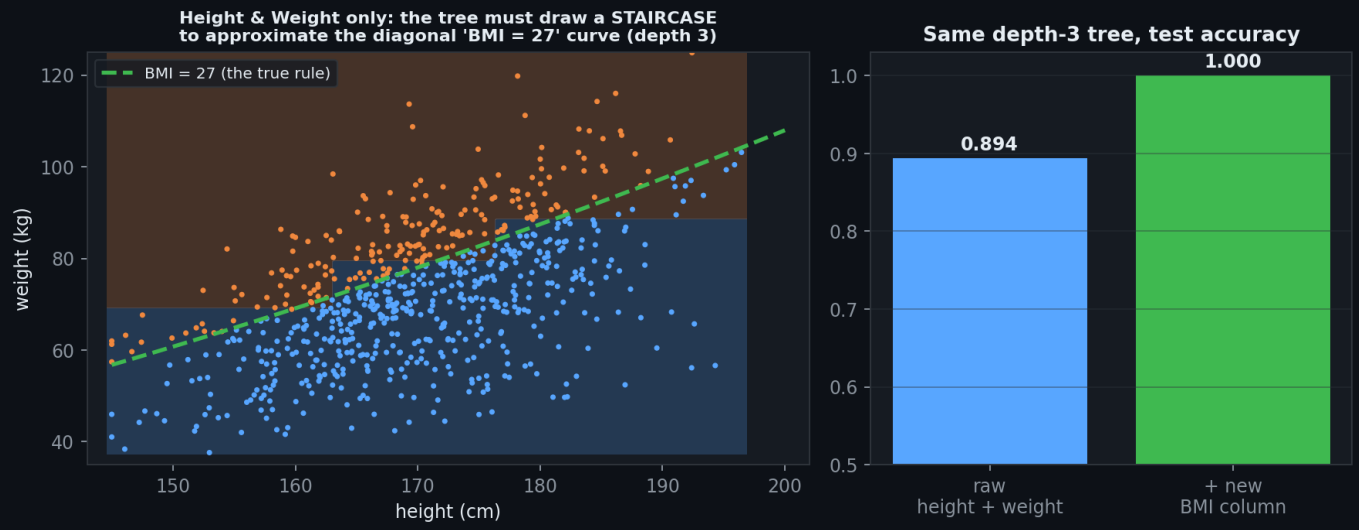


Figure 7.2 — Depth-3 tree, 700 synthetic people: height + weight only → 0.894 test accuracy; adding the BMI column → 1.000.

7.3 Balance class weights

Technical. If your dataset is highly skewed (e.g., 90 % normal transactions, 10 % fraud), the tree will aggressively optimise for the majority class to maximise overall accuracy. Set `class_weight='balanced'` in your model to force the algorithm to penalise mistakes on the rare class more heavily.

MATH How 'balanced' computes the weights

$$w_c = \frac{N}{K \cdot N_c}$$

N = total samples, K = number of classes, N_c = samples of class c. With 1 000 rows (900 normal, 100 fraud): $w_{\text{normal}} = 1000/(2 \cdot 900) = \mathbf{0.56}$, $w_{\text{fraud}} = 1000/(2 \cdot 100) = \mathbf{5.0}$. One fraud mistake now counts **9×** more in the impurity.

WATCH OUT The accuracy trap

A model that answers “normal” for every transaction is already 89 % accurate — and useless. Always check **recall** of the rare class, not only accuracy.

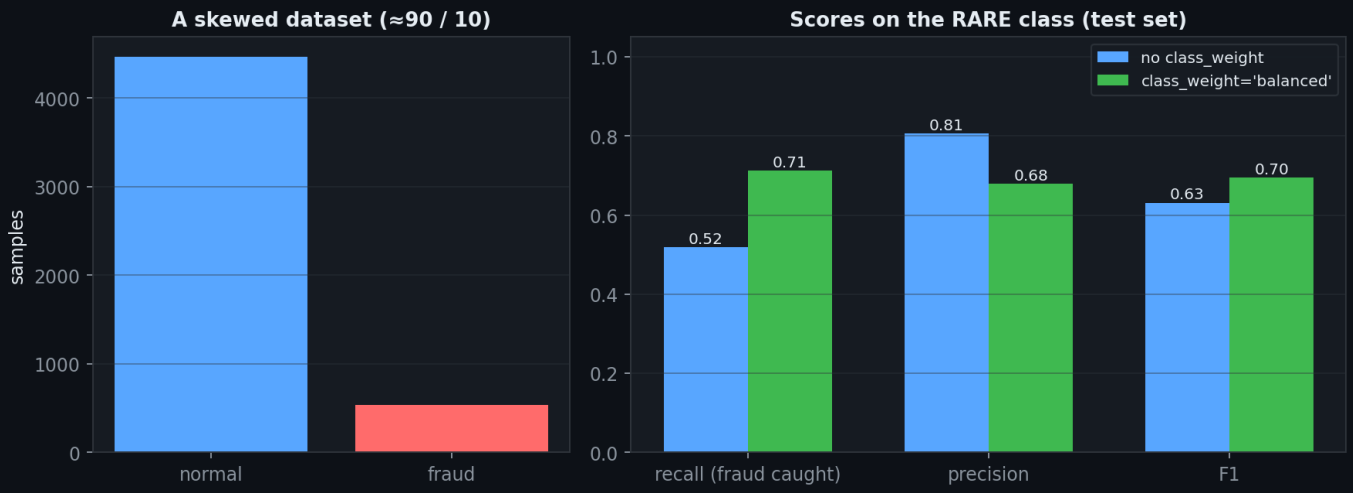


Figure 7.3 — Depth-6 tree on 5 000 synthetic transactions. Recall of the rare class rises from 0.52 to 0.71; precision falls — the usual trade-off.

7.4 Limit feature cardinality

Technical. Decision trees struggle with categorical variables that have dozens or hundreds of unique levels (like zip codes or product IDs). The tree will shatter the data into tiny, useless leaves. Group low-frequency categories into a single “Other” bucket or use target encoding before training.

IN PLAIN WORDS **Why it goes wrong**

A column with 300 different zip codes offers hundreds of ways to split the rows. With so many options, *some* split will look great purely by luck — the tree picks it, ends up with tiny leaves, and learns nothing that generalises.

zip_code (original)	count	→	zip_grouped
10001	420		10001
10002	380		10002
94110	7		**Other**
60601	3		**Other**
73301	1		**Other**

```
top = X['zip'].value_counts().head(20).index                  # keep only the 20 most frequent levels
X['zip_grouped'] = X['zip'].where(X['zip'].isin(top), 'Other')   # everything else -> 'Other'
```

MATH **Target encoding (smoothed mean of the target per category)**

$$\mathrm{TE}(c) = \frac{n_c\,\bar{y}_c + m\,\bar{y}}{n_c + m}$$

$\mathbf{n_c}$ = rows in category c, $\bar{\mathbf{y}}_\mathbf{c}$ = mean target in that category, $\bar{\mathbf{y}}$ = global mean, **m** = smoothing strength (rare categories are pulled toward the global mean). Compute it **inside each cross-validation fold**, otherwise the target leaks into the feature.

7.5 Drop highly correlated features

Technical. While decision trees handle collinearity better than linear regression, feeding them redundant features slows down training, increases memory usage, and makes the resulting tree diagram confusing and difficult to explain to stakeholders.

```
import numpy as np
corr = X.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if (upper[c] > 0.95).any()]   # nearly duplicate columns
X = X.drop(columns=to_drop)
```

EXAMPLE **A duplicate in disguise**

“Height in cm” and “height in inches” have correlation 1.0. The tree gains nothing from having both, importance gets split between them (each looks only half as important), and the diagram shows two interchangeable questions.

7.6 Remove noisy variables early (“junk data”)

Technical. Decision trees are “greedy” algorithms; they look for the best immediate split without considering the future. Feeding them random noise or irrelevant columns increases the chance they make a poor, hyper-specific split early on that cascades down the rest of the tree.

In plain words — “Remove junk data so the tree doesn't get distracted”. Decision trees are very short-sighted. They only look at the immediate next step without planning for the future. If you give the tree useless or repetitive columns, it might choose a bad path right at the beginning, which ruins the rest of the branches. Throw away confusing or duplicate data before you start.

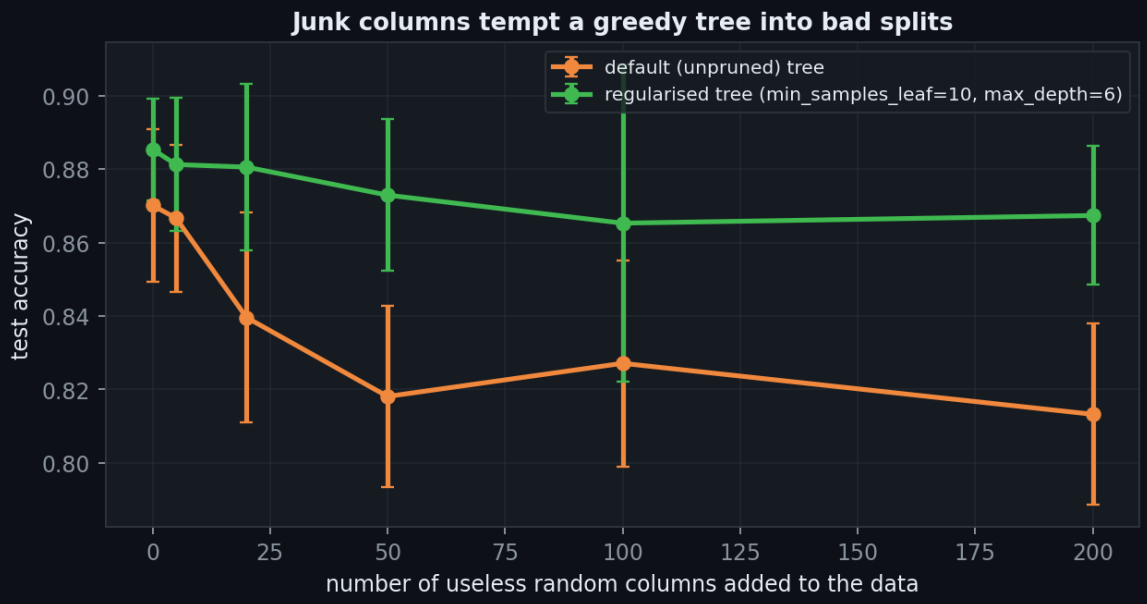


Figure 7.4 — Adding useless random columns to the two-moons data: the default tree drops from 0.87 to 0.81 test accuracy with 200 junk columns; a regularised tree suffers less (0.89 → 0.87). Mean of 8 runs.

Greedy means short-sighted: the XOR puzzle

In an XOR pattern the class depends on **two features together**; neither feature alone helps. The greedy tree sees no useful first question, so it picks an almost arbitrary one — here a sliver at the edge of the data — and wastes levels.

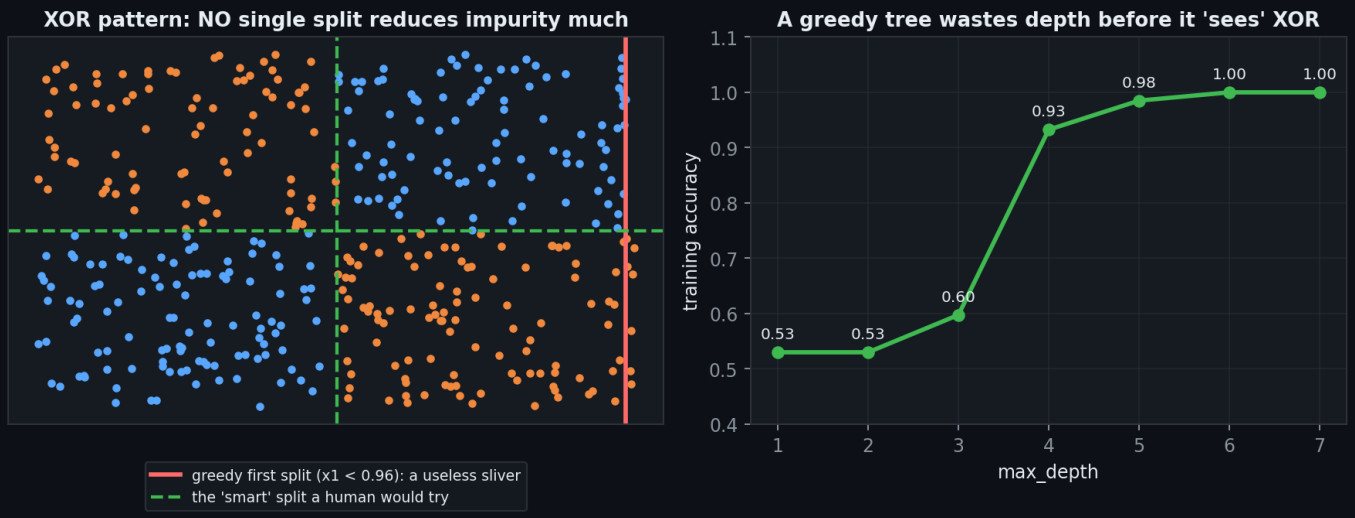


Figure 7.5 — Training accuracy needs depth 6 to reach 1.00, although two well-chosen splits (green) would almost solve it.

7.7 Hide some data to force creative thinking (max_features)

In plain words. If one piece of data is very strong (like “Salary” when predicting loan defaults), the tree will be lazy and just use that one column over and over. By telling the tree it is only allowed to look at a random handful of columns at each step (using `max_features`), you force it to look closer at the other data and find hidden patterns it would have normally ignored. (Figure 4.4 shows this effect measured.)

7.8 Put your whole computer to work (n_jobs=-1)

In plain words. When you are testing hundreds of different settings to find the perfect tree (Grid Search), your computer normally tests them one by one. By adding a simple piece of code like `n_jobs=-1`, you tell your computer to use all of its processors at the exact same time, finishing the testing process much faster.

```
GridSearchCV(dt_model, param_grid, cv=5, n_jobs=-1)   # n_jobs=-1  ->  use every CPU core in parallel
```

PRO TIP **Rule of thumb**

Time ≈ (number of fits × time per fit) ÷ number of cores. 1 920 small trees finish in seconds; 100 000 fits on big data may take hours — then switch to `RandomizedSearchCV`.

7.9 Accept that one tree is never enough (ensembles)

Technical. A single decision tree will almost always hit a hard ceiling on predictive efficiency because it is highly sensitive to small variations in the training data. Once you understand your data's behaviour in a single tree, the ultimate efficiency upgrade is to move to a **Random Forest** (which averages many trees to reduce variance) or **Gradient Boosting** (like XGBoost, which builds sequential trees to reduce bias).

In plain words — “Accept that one tree is never enough”. A single decision tree is like asking one person for advice; they will inevitably be a bit biased or memorise things too specifically. A single tree has a maximum limit on how accurate it can be. To get top-tier results, you have to upgrade to a “forest” (like a Random Forest or XGBoost model), which asks hundreds of trees for their opinion and combines the answers.

MATH **Why averaging helps, and how boosting differs**

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

Random Forest (top): B trees, each with variance σ² and pairwise correlation ρ. Averaging removes the second term as B grows; **max_features** makes trees different (lower ρ), so it shrinks the first term too. **Boosting (bottom):** each new small tree h_m is fitted to the mistakes of the current model F_{m-1} and added with a small step η (learning rate) — reducing bias step by step.

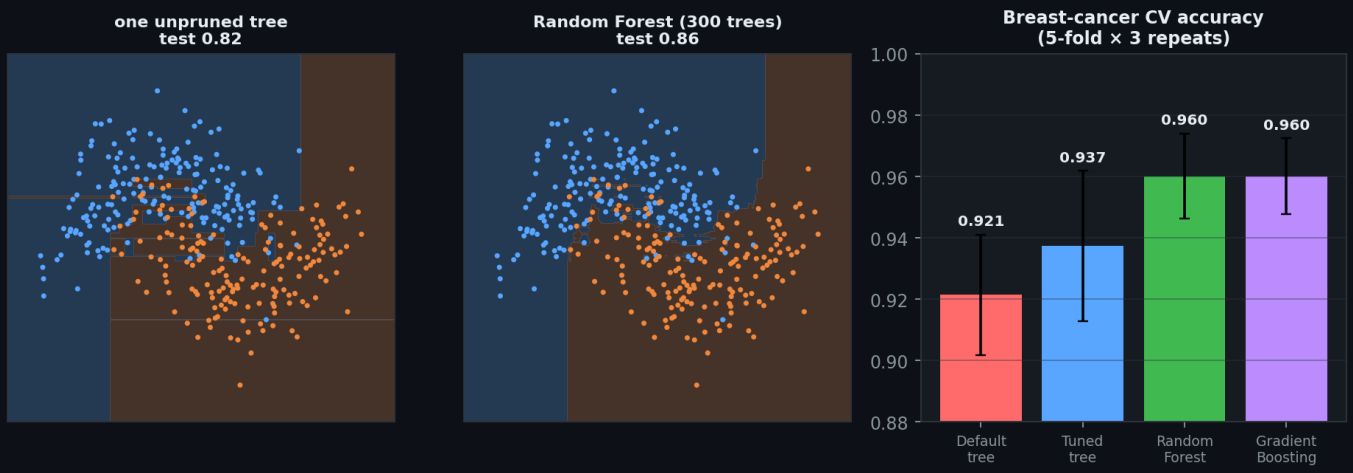


Figure 7.6 — Left/middle: one unpruned tree vs. a forest on two-moons data. Right: breast-cancer cross-validated accuracy (5-fold × 3 repeats): default tree 0.921, tuned tree 0.937, Random Forest 0.960, Gradient Boosting 0.960.

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
rf = RandomForestClassifier(n_estimators=300, max_features='sqrt', n_jobs=-1, random_state=42)
gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42)
# XGBoost / LightGBM follow the same fit/predict pattern:  xgboost.XGBClassifier(...)
```

KEY IDEA **Your tuned tree is not wasted**

Everything you learned (max_depth, min_samples_leaf, max_features, criterion …) reappears inside forests and boosting. In a forest you usually grow **deep, unpruned** trees and tune `n_estimators` and `max_features`; in boosting you keep trees **shallow** (depth 3–6) and tune `learning_rate`.

PRO · CHAPTER 8

8 Pro-Level Extras, Cheat-Sheet & Practice

A professional workflow, interpretation tools, the symptom → fix table, a glossary and exercises.

8.1 A professional workflow in eight steps

- **Split first** (stratified for classification) and lock the test set away.
- **Baseline:** train the default tree and a dummy model; record train *and* validation scores.
- **Clean the inputs:** drop duplicates/correlated/noisy columns, group rare categories, add smart features (Chapter 7).
- **Pick a metric** that matches the business (accuracy, F1, recall, ROC-AUC).
- **Tune** with cross-validation: coarse grid → fine grid, or ccp_alpha path.
- **Prefer the simplest** model whose score is within one standard deviation of the best.
- **Evaluate once** on the test set; inspect errors and feature importances.
- **Compare with an ensemble.** If it is clearly better and you can afford the loss of interpretability, ship the ensemble.

8.2 Putting preprocessing inside a Pipeline (no leakage)

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

pre  = ColumnTransformer([('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)], remainder='passthrough')
pipe = Pipeline([('prep', pre), ('clf', DecisionTreeClassifier(random_state=42))])

params = {'clf__max_depth': [None, 5, 10], 'clf__min_samples_leaf': [1, 5, 10], 'clf__ccp_alpha': [0.0, 0.005, 0.01]}
search = GridSearchCV(pipe, params, cv=5, n_jobs=-1, scoring='f1').fit(X_train, y_train)
```

Because the encoder lives **inside** the pipeline, it is re-fitted on the training part of every fold — information from the validation fold never leaks. Note the `clf__` prefix: it routes the parameter to the tree.

8.3 Interpreting a tree

```
from sklearn.tree import export_text
print(export_text(best_tree, feature_names=list(X.columns), max_depth=3))   # rules as plain text
```

MATH **Feature importance (Gini / “mean decrease in impurity”)**

$$\text{Imp}_j \propto \sum_{t\,:\,\text{split on } j} \frac{N_t}{N}\,\Delta I_t \qquad \sum_j \text{Imp}_j = 1$$

For every node t that splits on feature j, take the impurity decrease ΔI_t weighted by the share of samples N_t/N reaching it; add them up and normalise. Caveats: it favours features with many distinct values and is computed on training data — cross-check with `sklearn.inspection.permutation_importance` on the test set.

8.4 Choosing the scoring metric

Situation	Use scoring=	Why
Balanced classes	`'accuracy'`	Simple and fair when classes are similar in size.
Imbalanced (fraud, disease)	`'f1'` / `'recall'` / `'average_precision'`	Accuracy hides failures on the rare class.
Ranking quality / probabilities	`'roc_auc'`	Threshold-independent.
Regression	`'neg_mean_squared_error'` / `'r2'`	Scikit-learn maximises scores, so errors are negated.

8.5 Classic mistakes

Mistake	Consequence	Fix
Tuning on the test set	Over-optimistic score (leakage)	Tune with CV inside the training set; touch the test set once
No random_state	Results change every run	Set random_state everywhere
Scaling for a tree	Wasted effort, no gain	Skip it (but keep it for k-NN, SVM …)
Judging by accuracy on imbalanced data	Useless model looks great	Use class_weight + recall/F1
Trusting one small test split	Noise mistaken for improvement	Repeated cross-validation
Fitting encoders/imputers before splitting	Leakage	Put them in a Pipeline
Huge grids by default	Hours of compute	Coarse→fine, RandomizedSearchCV, ccp_alpha path

8.6 Cheat-sheet: symptom → remedy

What you observe	Diagnosis	Try
Train ≈ 100 %, test much lower	Overfitting (high variance)	↓ max_depth, ↑ min_samples_leaf / split, ↑ ccp_alpha, use max_features='sqrt', or a forest
Both train and test low	Underfitting (high bias)	↑ max_depth, ↓ min_samples_leaf, ↓ ccp_alpha, engineer better features, try boosting
Great accuracy, rare class ignored	Imbalance	class_weight='balanced', F1/recall scoring, resampling
Tree is huge and unreadable	Too complex	ccp_alpha, max_leaf_nodes, drop junk features
Results differ run to run	Randomness / tiny data	random_state, repeated CV, more data
Grid search takes forever	Too many combinations	n_jobs=-1, RandomizedSearchCV, smaller grid
Accuracy plateau at ~90–95 %	Single-tree ceiling	Random Forest / Gradient Boosting / XGBoost

8.7 Glossary

Term	Meaning
Bias	Error from a model being too simple.
Variance	Error from a model reacting too strongly to the particular training sample.
Overfitting	Memorising noise: great train score, poor test score.
Pruning	Removing branches after (post-pruning) or preventing them while (pre-pruning) the tree grows.
Cross-validation	Averaging validation scores over k train/validation splits.
Grid search	Trying every combination of listed hyperparameter values.
Ensemble	Combining many models into one prediction.
Greedy algorithm	Chooses the best option right now without planning ahead.
Information gain	Drop in entropy produced by a split.
Leakage	Information from validation/test data sneaking into training.
Cardinality	Number of distinct values of a categorical feature.
Collinearity	Two features carrying nearly the same information.

8.8 Practice exercises

- **By hand:** change the label of the 47-year-old customer to “Buy”. Recompute the weighted Gini for every threshold in Section 2.4. Which split wins now?
- **Depth curve:** on any dataset, plot train and test accuracy for max_depth = 1…20 and find the sweet spot.
- **Leaf rule:** repeat with min_samples_leaf = 1…50. Which of the two parameters gives a smoother curve?
- **Pruning path:** compute `cost_complexity_pruning_path`, plot test accuracy against alpha, and pick the best alpha with cross-validation.
- **Imbalance:** create a 95 / 5 dataset with `make_classification`; compare recall with and without class_weight.
- **Forest vs tree:** cross-validate your tuned tree against a Random Forest on your own data. How big is the gap?
- **Feature engineering:** add one hand-made ratio feature to a dataset (like BMI) and measure the change.

KEY IDEA **Key takeaways**

1) A tree is a learned flow-chart that greedily picks the question with the biggest impurity drop.
2) Left alone, it memorises noise — hyperparameters (**max_depth, min_samples_split, min_samples_leaf, max_features, ccp_alpha, criterion**) limit its complexity.
3) GridSearchCV + cross-validation finds good settings; compare fairly and keep the test set untouched.
4) Skip scaling, balance classes, tame categories, remove junk, engineer smart features.
5) When a single tree hits its ceiling, move to Random Forest or Gradient Boosting — they are built from the trees you now understand.